

Computing Center, IHEP, CAS
National HEP Data Center

Tutorials and Practice of OpenDrSai Framework and AI Agents for Scientific AI Agent

Dongbo Xiong

On behalf of HepAI Group



◆ 1.Install OpenDrSai Package:

- Command : conda create -n drsai python=3.12
- Command : pip install drsai drsai_ui -U

◆ 2.Git OpenDrSai code from github or gitlab to obtain examples:

- Command: git clone <https://github.com/hepai-lab/drsai>
- Command: git clone <https://code.ihep.ac.cn/hepai/drsai>

◆ 3.Get RAGFlow :

- <https://ragflow.ihep.ac.cn>
- Get RAGFlow API from <https://aiweb01.ihep.ac.cn:886/user-setting/api>

◆ 4.Linux or WSL is better !!!

Outline

01 Introduction

02 Quick Start

03 How to build an AI Agent with OpenDrSai

04 What Is a Multi-Agent System

05 How to Build Your Own Scientific AI Agent

01-Introduction

What is AI Agent?

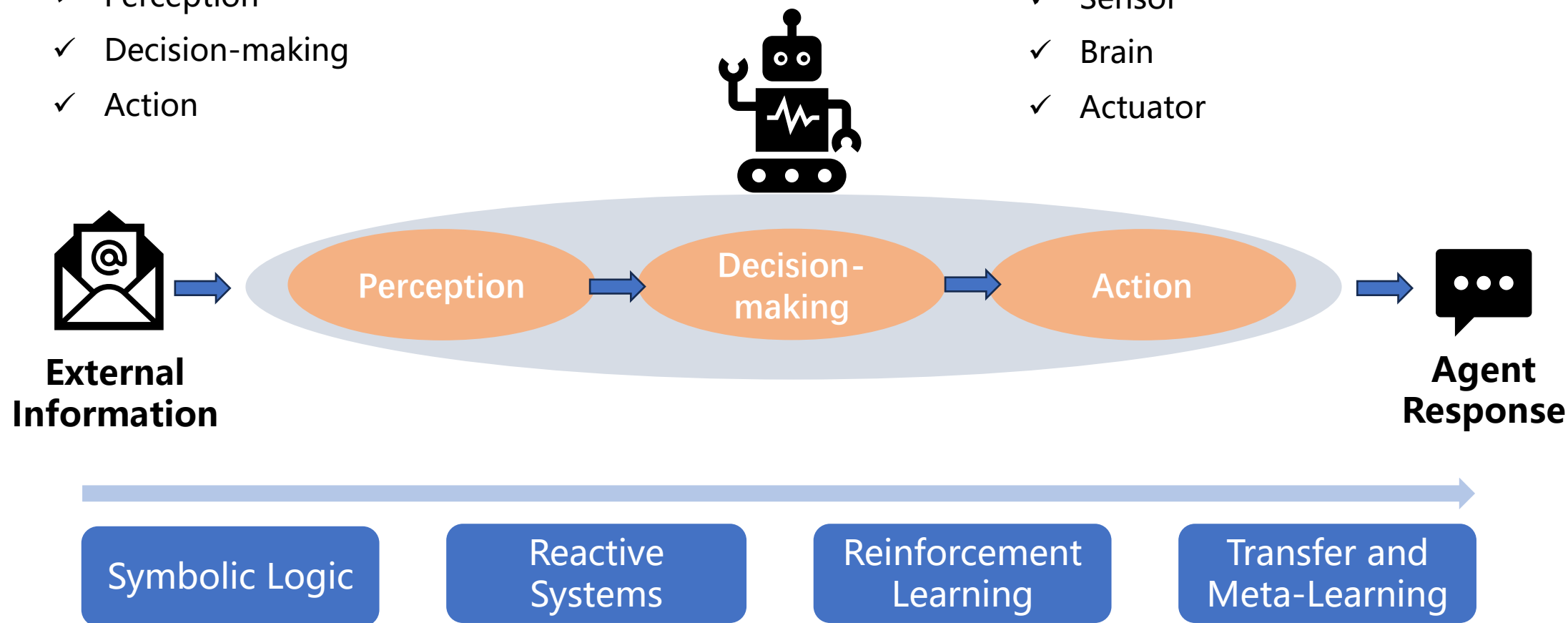


◆ Core Concepts of AI Agent:

- ✓ Perception
- ✓ Decision-making
- ✓ Action

◆ Core Components of AI Agent:

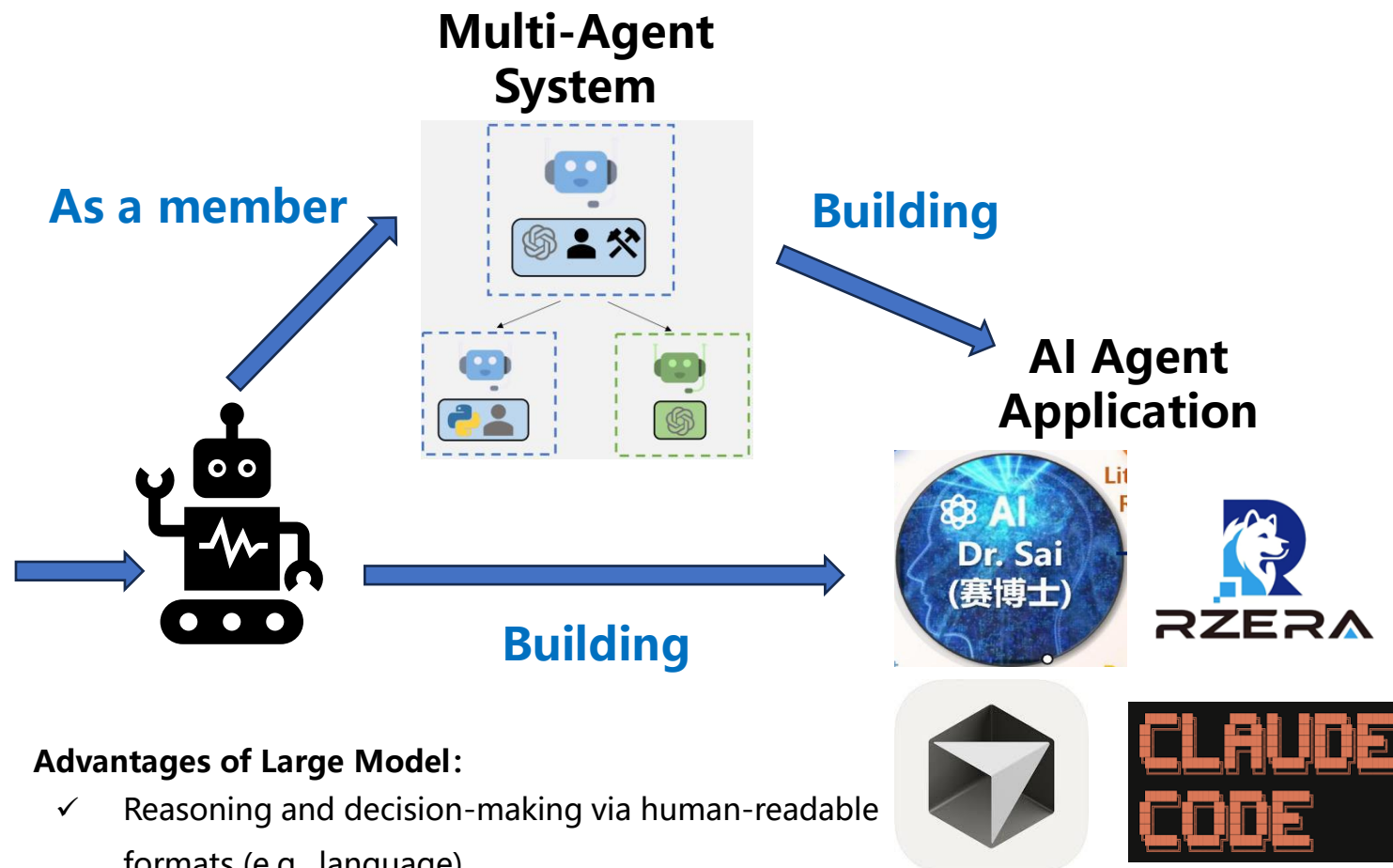
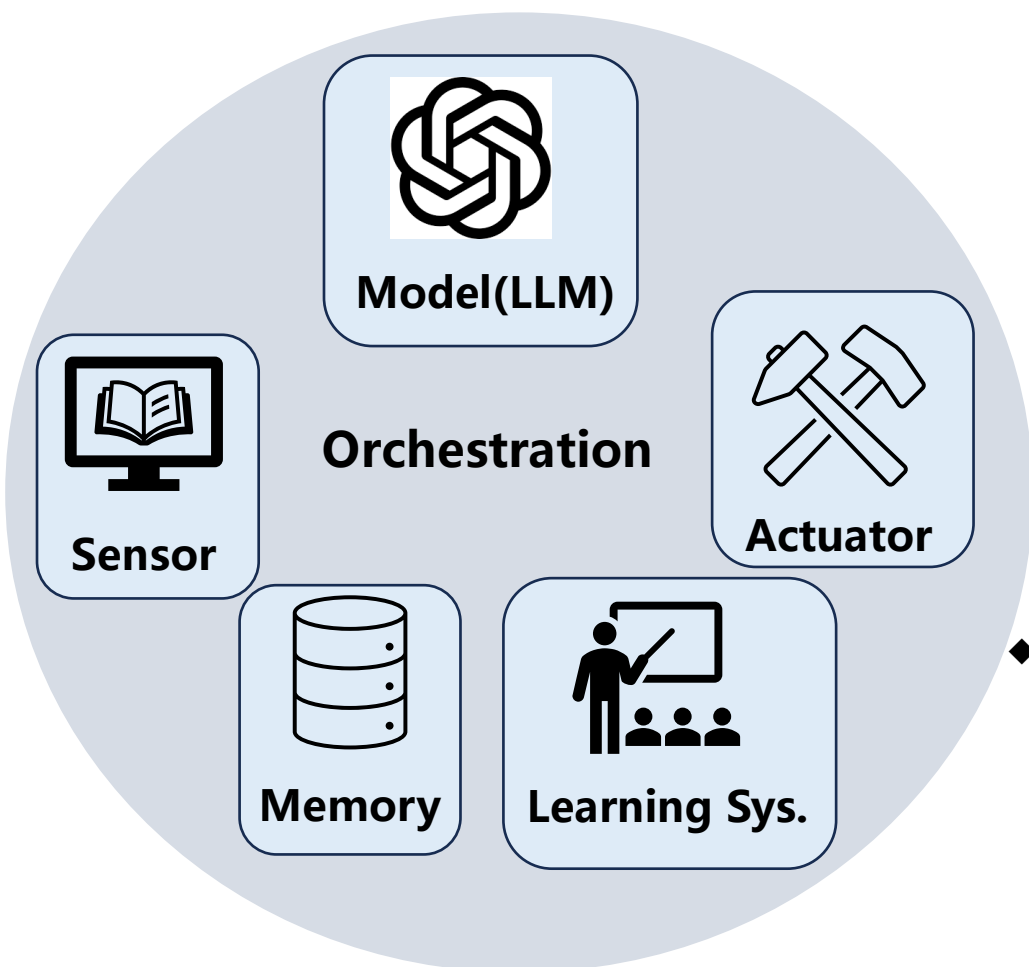
- ✓ Sensor
- ✓ Brain
- ✓ Actuator



What is AI Agent?



◆ Large Model-based Agents:



◆ Advantages of Large Model:

- ✓ Reasoning and decision-making via human-readable formats (e.g., language)
- ✓ Ability to learn and adapt using external knowledge and feedback
- ✓ ...

AI Agent Application

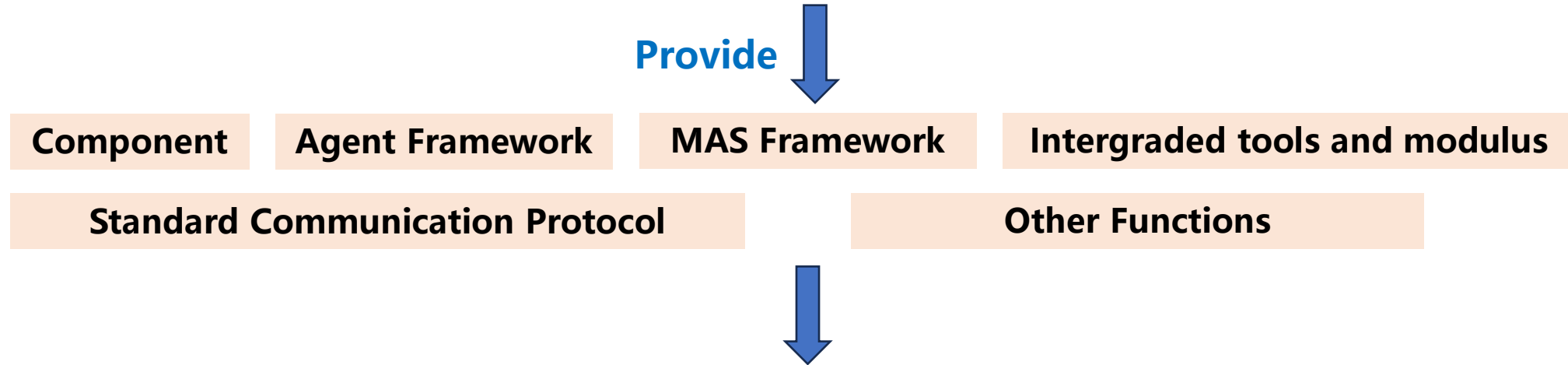


Level	Name	Key Characteristics	Capability Boundaries	Example Applications
Level 1	Basic Responder	- Passive input-output - No memory or decision-making - Rule-based or pretrained models	- Handles only single-step, predefined tasks - No tool use or context awareness	- Simple FAQ chatbots - Symptom lookup tools (e.g., early MYCIN-like systems)
Level 2	Routing Agent	- Selects from pre-defined paths/functions - Uses decision trees or classifiers	- Multi-step but fixed workflows - Cannot create new tools or logic	- Smart triage in hospitals - Bank onboarding flow guides
Level 3	Tool-Augmented Agent	- Dynamically decides when/how to call external tools - Sets parameters autonomously	- Solves complex tasks using APIs/tools - Limited to known toolset & rules	- AI radiology assistants (e.g., Enlitic) - Real-time fraud detection in finance
Level 4	Multi-Agent System	- Orchestrates multiple specialized sub-agents - Goal-driven coordination	- Handles large-scale, cross-domain problems - Sensitive to network latency & sync	- Federated medical diagnosis (e.g., Rhino Health) - Huawei's intelligent healthcare platform
Level 5	Autonomous Agent	- Self-generates and executes code - Learns and iterates independently	- Theoretically general-purpose - Constrained by safety, reliability, and regulation	- StockGPT: auto-generates trading strategies - Experimental AI clinicians generating treatment plans

What is AI Agent Development Framework?



- An **Agent Development Framework** is an integrated system that provides the **fundamental infrastructure, standardized interfaces, and runtime environment** for constructing, executing, coordinating, and managing AI agents.



◆ Why we need AI Agent Development Framework:

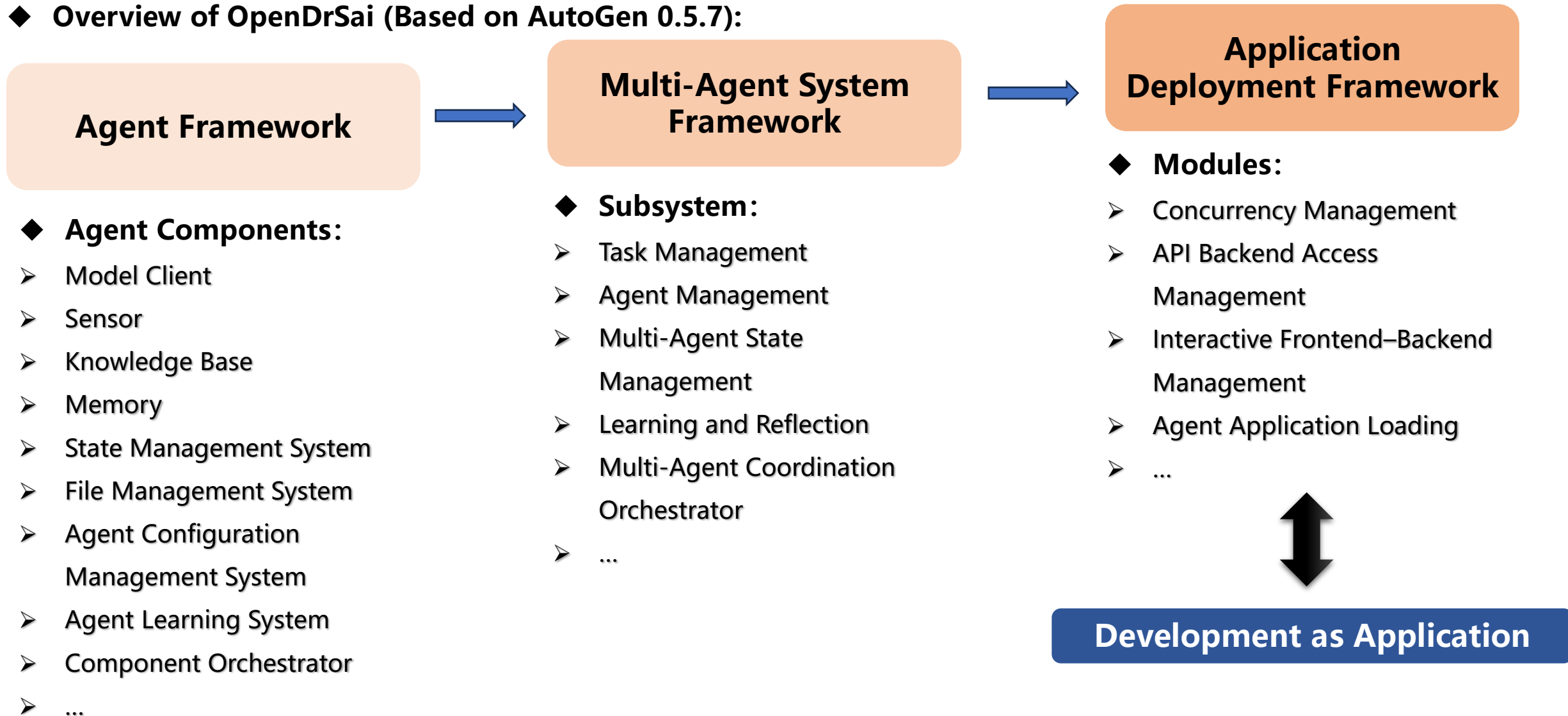
- **Accelerated Development** – Pre-built components (agents, tools, memory, etc.) reduce boilerplate code and speed up prototyping.
- **Built-in Collaboration Logic** – Frameworks handle communication, task delegation, and coordination between agents.
- **Tool & API Integration** – Easy integration with external tools, databases, and APIs (e.g., search, code exec, LLMs).
- **Modularity & Reusability** – Agents and workflows can be reused across projects with minimal changes.
- **Community & Ecosystem** – Leverage documentation, examples, and community plugins to solve common problems faster
- **Memory & State Management** – Built-in support for short-term and long-term memory across interactions.
- ...

AI Agent Development Framework



Framework	Design Philosophy	Collaboration Mechanism	Typical Use Cases
Dify	Low-code/no-code enterprise AI platform	Tool-chain orchestration	Knowledge Q&A, customer support, MVP apps
AutoGen	Dialogue-driven multi-agent collaboration	Conversational interaction	Research, code generation, complex workflows
CAMEL-AI	Role-playing autonomous agents	Role-based task delegation	Academic research, document collaboration
CrewAI	Simulated human team coordination	Hierarchical task orchestration	Content creation, data analysis, marketing
LangChain	Modular, chain-based LLM application dev	Tool calling (mostly single-agent)	RAG, document QA, custom LLM pipelines
Google Agent (ADK)	Code-first modular agent toolkit	Agent-to-Agent (A2A) protocol	Enterprise automation, multimodal streaming

◆ Overview of OpenDrSai (Based on AutoGen 0.5.7):





◆ Why Choose OpenDrSai:

◆ AI Ecosystem:

- ✓ We provide a complete ecosystem for AI scientific agent development:
 - **Large models** (HepAI, Aliyun/Qwen, OpenAI, Deepseek, 磐石, Anthropic, Baichun, Bytedance, Minimax...),
 - **Knowledge bases** (RAGFlow, Dify, LightRAG, Qdrant/Llama_index...)
 - **Database** (PostgreSQL...) and **Management System**(DOMAS...)
 - **Tools integration** and **long task management** (MCP, OpenAPI...)
 - **Deployment and scheduling platform** (OpenAI API, A2A, OpenWebUI, Chainlit, DrSai UI, Qionwu...)
 - ...

◆ Practice in development for Scientific Agent

- ✓ We also have extensive experience in integrating professional scientific tools and developing domain-specific scientific agents such as Dr.Sai and DataAgent.

◆ Open-Source Development:

- ✓ Developed through the enhancement of open-source general frameworks, OpenDrSai balances general compatibility with the specialized requirements of scientific AI agent development.



◆ Applicable Scenarios for AI Agents:

- ✓ **Complex Decision-Making:** Workflows that involve detailed judgments, exceptions, or context-dependent decisions — for example, tasks requiring multi-turn contextual understanding and intent recognition.
- ✓ **Hard-to-Maintain Rules:** Scenarios where large and complex rule sets make the system difficult to manage, costly to update, or prone to errors — such as workflow safety reviews.
- ✓ **High Dependence on Unstructured Data:** Situations requiring interpretation of natural language, extraction of meaning from documents, or conversational interaction with users — for example, processing research papers or other unstructured data.

02-Quick Start

2.1.Starting with an Agent with MCP Tools



◆ MCP Server example is in `examples/agent_groupchat/mcp_sse_server.py`

1. Start a MCP tool SSE server: `python mcp_sse_server.py`.

```
from mcp.server.fastmcp import FastMCP

# Create an MCP server
mcp = FastMCP(
    name="Calculator",
    instructions="A simple calculator!",
    host="0.0.0.0",
    port=42608,
)

# Add an addition tool
@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

if name == " main ":
    mcp.run(transport="sse")
```

1.1.Configure the MCP service name, description, and the port on which it will run.

1.2.Use @mcp.tool to annotate the functions you want to expose as callable tools.

1.3.Choose to start the MCP remote service using SSE.

2. Test it within `examples/agent_groupchat/mcp_sse_server_test.py`

2.1.Starting with an Agent with MCP Tools



```
# Create a factory function to ensure isolated Agent instances for concurrency
async def create_agent() -> AssistantAgent:
```

```
# Create model client
model_client = HepAIChatCompletionClient(
    model="deepseek-ai/deepseek-v3-1",
    base_url="https://aiapi.ihep.ac.cn/apiv2",
    api_key=os.getenv('HEPAI_API_KEY'),
    model_info={
        "vision": False,
        "function_calling": True, # You must sure that the model
        "json_output": True,
        "structured_output": False,
        "family": ModelFamily.GPT_41,
        "multiple_system_messages": True,
    },)
```

```
tools = await mcp_server_tools(SseServerParams(
    url="http://0.0.0.0:42608/sse",
    env=None))
```

```
# Create assistant agent with the model client and memory
assistant_agent = AssistantAgent(
    name="assistant_agent",
    system_message="""你是一个可以使用简单计算器进行加法计算的助手。",
    description="一个简单计算器助手",
    model_client=model_client,
    model_client_stream=True,
    tools=tools,
)
```

```
return assistant_agent
```

3.Add MCP tools to Agent

3.1.Create a ModelClient Component

◆ MCP Agent example is in [examples/agent_groupchat/assistant_tool_mcp_sse.py](#)

3.2.Access remote MCP server

3.3.Create an Agent using the above components

4.Wrap the agent inside a function

2.1.Starting with an Agent with MCP Tools



4. Test the MCP Agent in console mode

```
asyncio.run(  
    run_console(  
        agent_factory=create_agent,  
        task="-5+20=?"  
    )  
)
```

◆ MCP Agent example is in [examples/agent_groupchat/assistant_tool_mcp_sse.py](#)

5. Register as an API server

```
asyncio.run(  
    run_worker(  
        # 智能体注册信息  
        agent_name="Calculate_Assistant",  
        author = "xiongdb@ihep.ac.cn",  
        permission='groups: drsai, payg, ddf_free; users: admin, xiongdb@ihep.ac.cn, ddf_free, yqsun@ihep.ac.cn; owner: xiongdb@ihep.ac.cn',  
        description = "一个简单计算器助手",  
        version = "0.1.0",  
        logo="https://aiapi.ihep.ac.cn/apiv2/files/file-8572b27d093f4e15913bebfac3645e20/preview",  
        # 智能体实体  
        agent_factory=create_agent,  
        # 后端服务配置  
        port = 42816,  
        no_register=False,  
        enable_openwebui_pipeline=True,  
        history_mode = "backend",  
        # use_api_key_mode = "backend",  
    )  
)
```

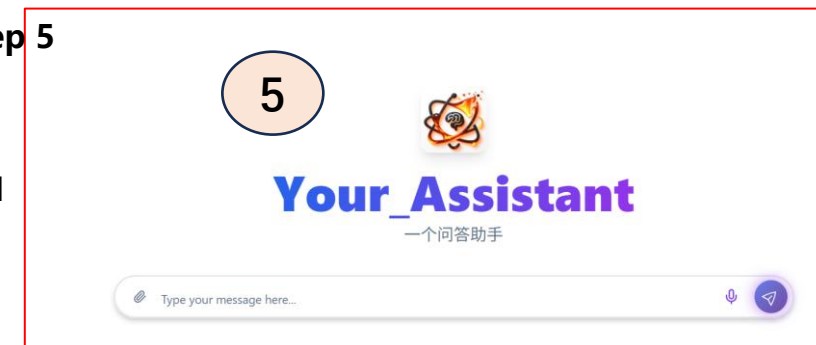
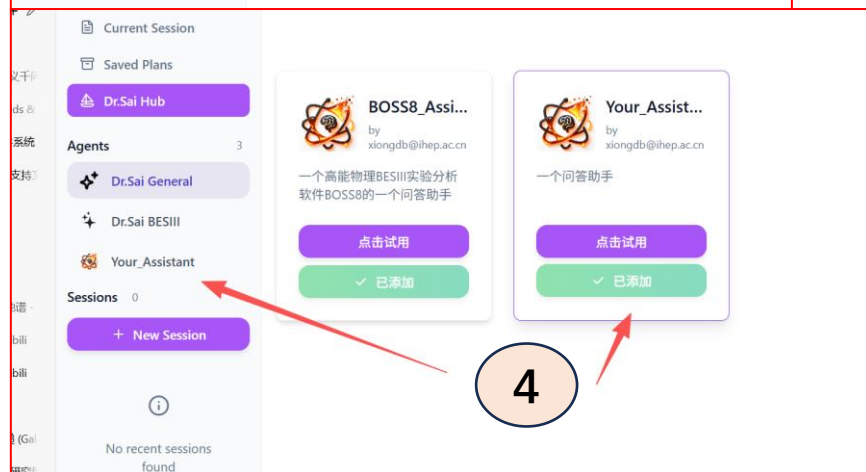
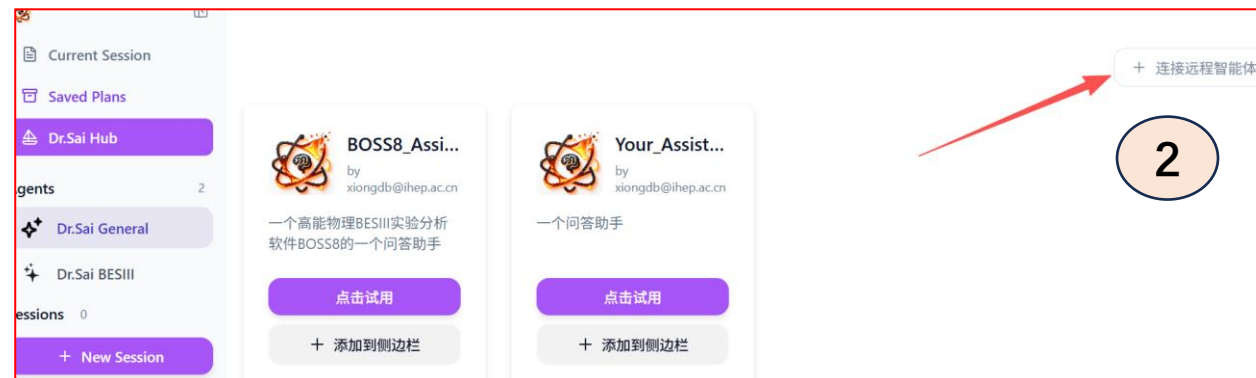
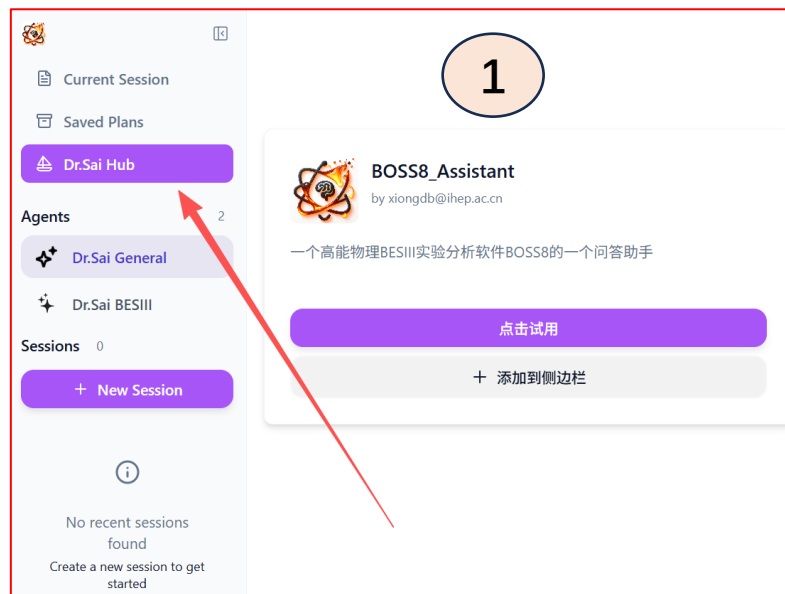
2.1.Starting with an Agent with MCP Tools



6.Start a UI for Agent: drsai ui



Default url is <http://localhost:8081>, Use `--port xxxx` assign other port



2.2.Starting with an Agent with Specialized Knowledge



◆ Example is in `examples/agent_groupchat/assistant_momery.py`

1.Create a ModelClient Component

```
# Create model client
model_client = HepAIChatCompletionClient(
    model="deepseek-ai/deepseek-v3-1",
    model_info={
        "vision": False,
        "function_calling": True, # You
        "json_output": True,
        "structured_output": False,
        "family": ModelFamily.GPT_41,
        "multiple_system_messages": True,
    },
)
```

3.Create a ModelContext Component

```
# Create model context
model_context = TokenLimitedChatCompletionContext(
    model_client=model_client,
    token_limit=100000,
)
```

5.Wrap the agent inside a function

6. Register as an API server

7.Start a UI for Agent

2.Create a Momery Component and add knowledge

```
with open("examples/agent_groupchat/Your_specific_konwledge.md", "r") as f:
    Your_specific_konwledge = f.read()

list_momery = ListMemory(name = "list_memory")
await list_momery.add(
    content=MemoryContent(
        content=Your_specific_konwledge,
        mime_type=MemoryMimeType.TEXT,
    )
)
```

4.Create an Agent using the above components

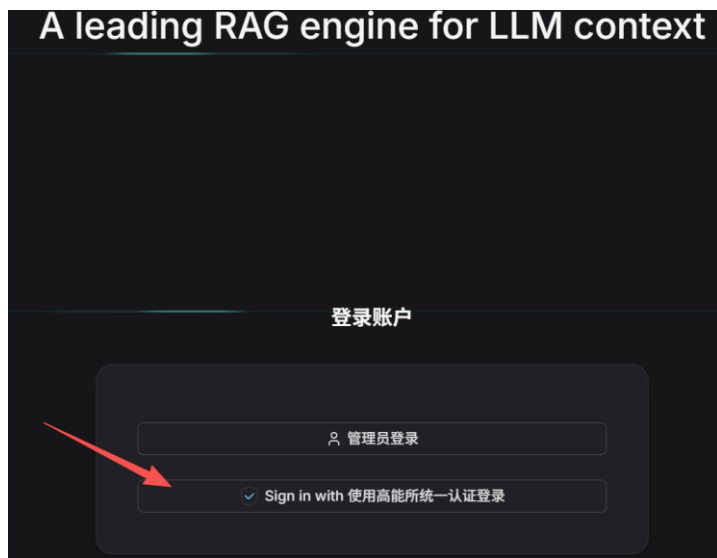
```
# Create assistant agent with the model client and memory
assistant_agent = AssistantAgent(
    name="assistant_agent",
    system_message="""你是一个问答助手，需要根据检索到的记忆内容进行回复。""",
    description="一个问答助手",
    model_client=model_client,
    model_client_stream=True,
    memory=[list_momery],
    model_context=model_context
)
```

2.3.Starting with an Agent with RAGFlow

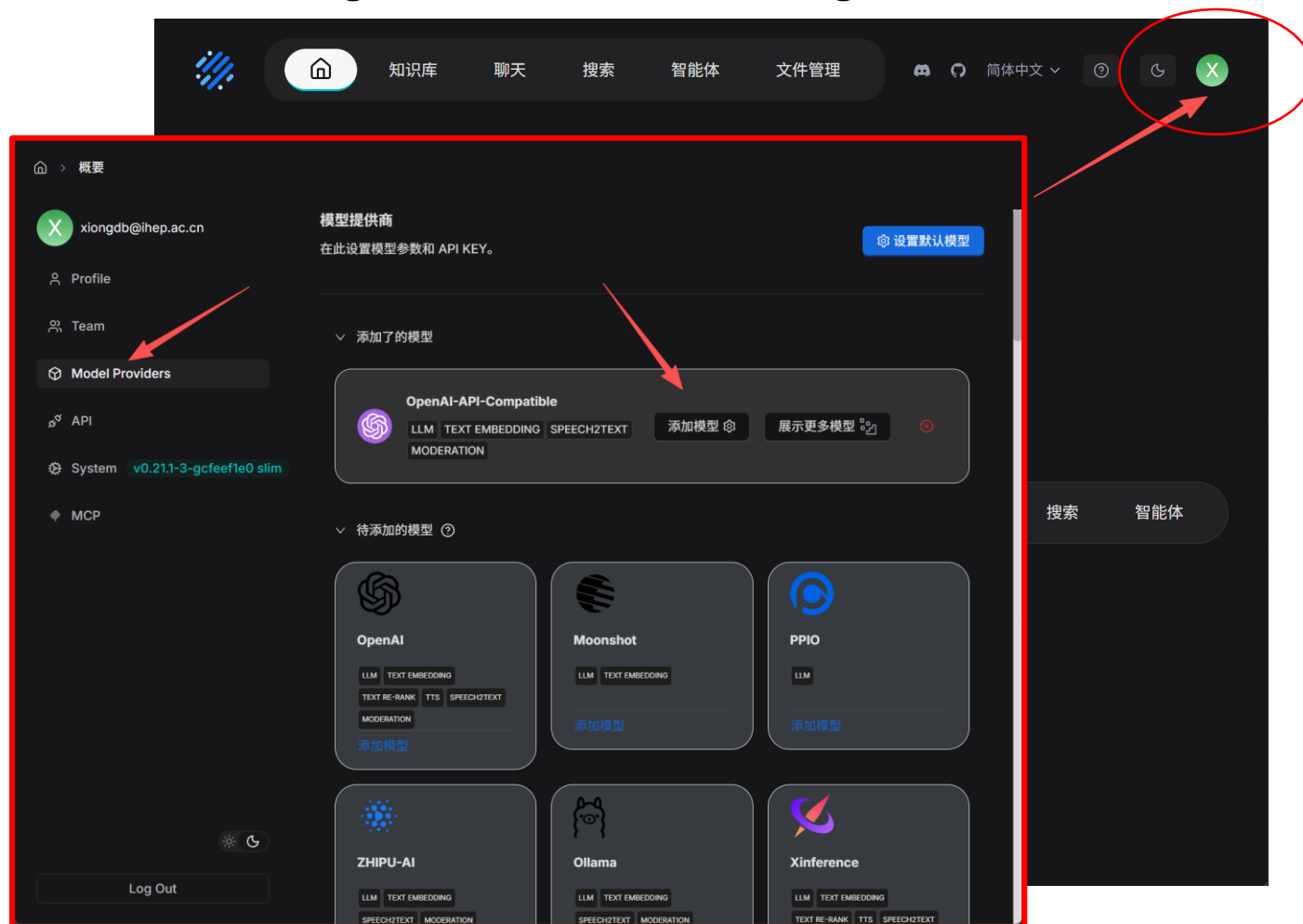


- ◆ Use RAGlow: <https://ragflow.ihep.ac.cn/login>

1.Login to the RAGFlow server



2. Configure the chat, embedding and reranker models



2.3.Starting with an Agent with RAGFlow



◆ Use RAGlow: <https://ragflow.ihep.ac.cn/login>

2. Configure the chat, embedding and reranker models

编辑 OpenAI-API-Compatible 模型

* 模型类型

chat

* 模型名称

deepseek-ai/deepseek-v3

* 基础 Url

https://aiapi.ihep.ac.cn/apiv2

API-Key

请输入api key (如果是本地部署的模型, 请忽略它)

* 最大token数

100000

是否支持 Vision

☐

如何集成 OpenAI-API-Compatible

取消

确定

编辑 OpenAI-API-Compatible 模型

* 模型类型

embedding

* 模型名称

hepai/bge-m3:latest

* 基础 Url

https://aiapi.ihep.ac.cn/apiv2

API-Key

请输入api key (如果是本地部署的模型, 请忽略它)

* 最大token数

4000

如何集成 OpenAI-API-Compatible

取消

确定

添加 LLM

* 模型类型

rerank

* 模型名称

hepai/bge-reranker-v2-m3:latest

* 基础 Url

https://aiapi.ihep.ac.cn/apiv2

API-Key

请输入api key (如果是本地部署的模型, 请忽略它)

* 最大token数

1024

如何集成 OpenAI-API-Compatible

取消

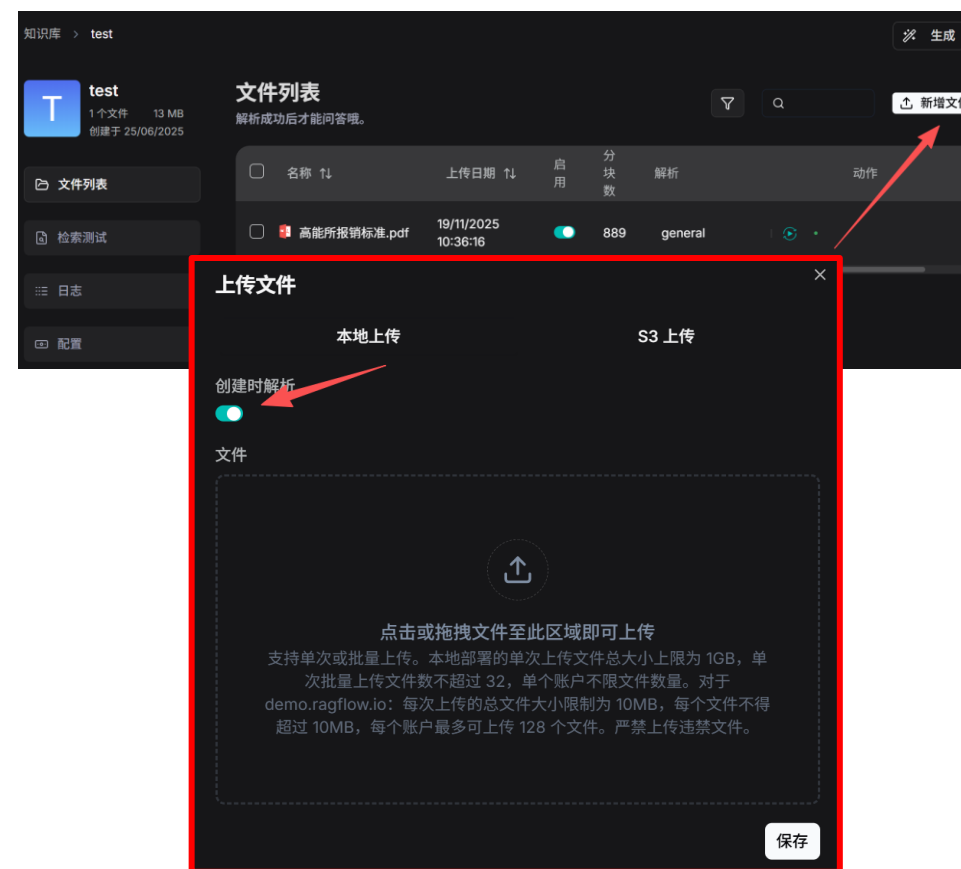
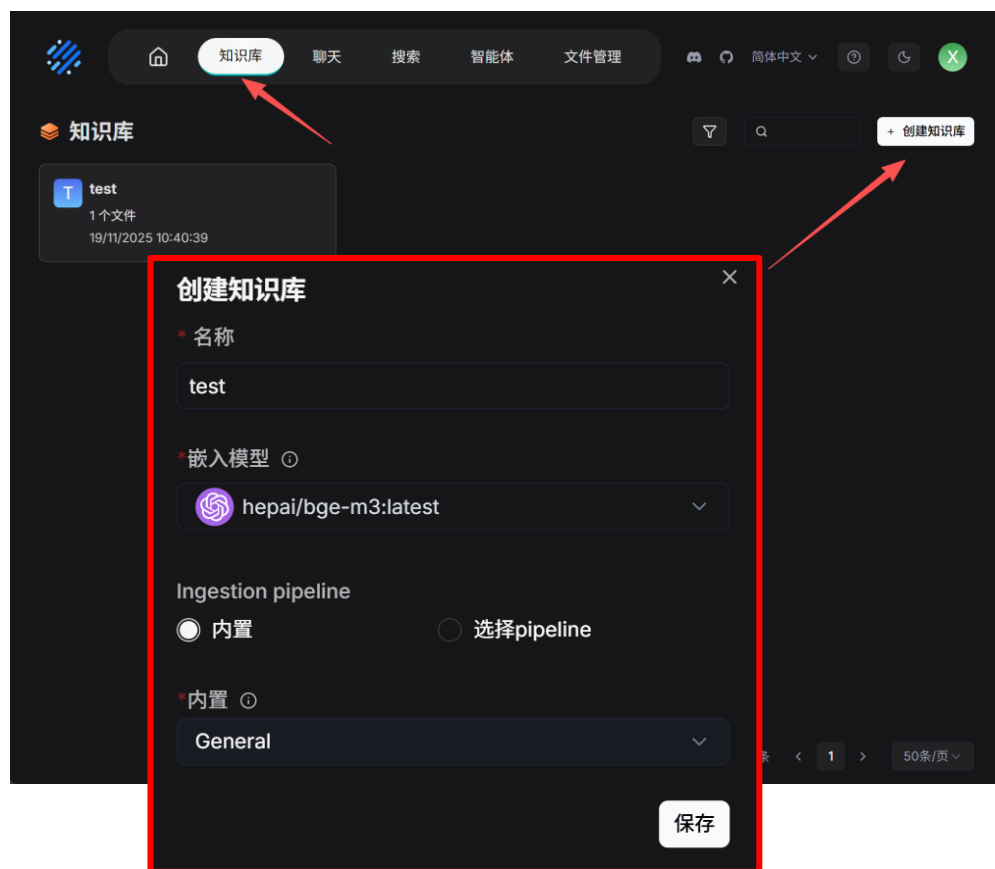
确定

2.3.Starting with an Agent with RAGFlow



- ◆ Use RAGlow: <https://ragflow.ihep.ac.cn/login>

3.Create your knowledge base

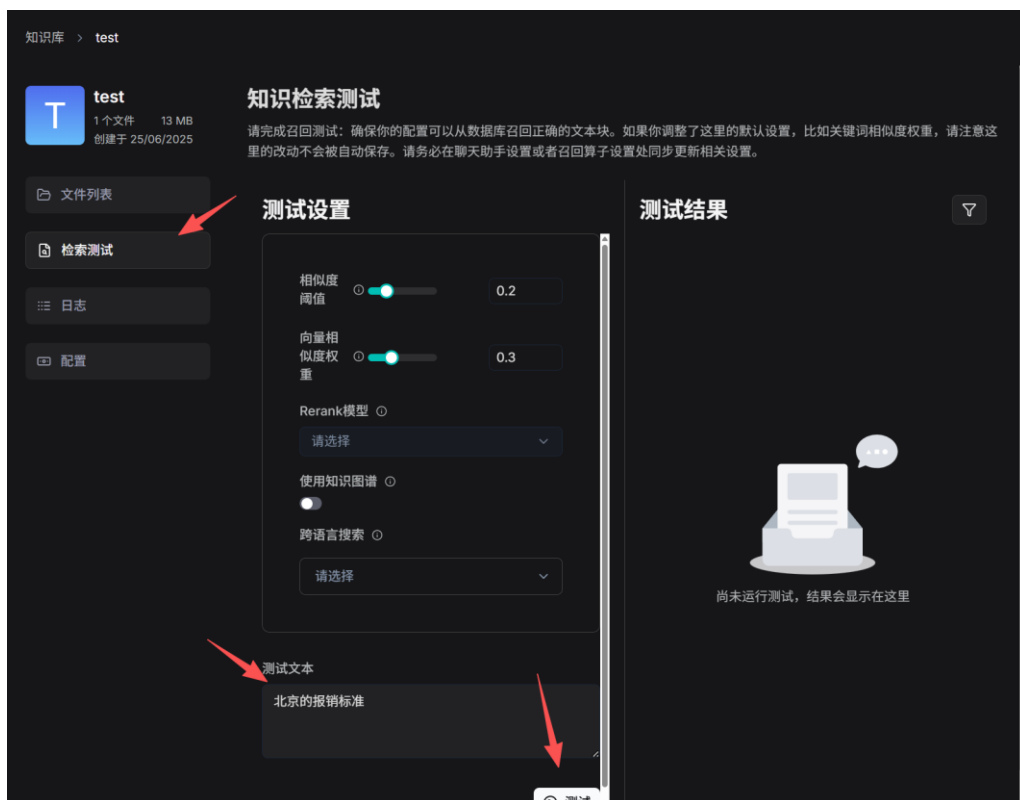


2.3.Starting with an Agent with RAGFlow



◆ Use RAGlow: <https://ragflow.ihep.ac.cn/login>

4. Test retrieval performance



2.3.Starting with an Agent with RAGFlow



5.Get the RAGFlow API

知识库

RAGFlow API

API KEY

API 服务器: <https://ragflow.ihep.ac.cn>

HTTP API

A complete reference for RAGFlow's RESTful API. Before proceeding, please ensure you have a RAGFlow API key ready for authentication.

API DOCS

Code	Message	Description
400	Bad Request	Invalid request parameters
401	Unauthorized	Unauthorized access
403	Forbidden	Access denied
404	Not Found	Resource not found
500	Internal Server Error	Server internal error
1001	Invalid Chunk ID	Invalid Chunk ID

```
async def list_ragflow_datasets(  
    page: int = 1,  
    page_size: int = 10,  
    orderby: str = "create_time",  
    desc: bool = True,  
    name: str = None,  
    dataset_id: str = None  
):  
    """  
    List all available RAGFlow datasets.  
    """  
    desc: Sort in descending order (default: True)
```

List all available RAGFlow datasets using examples/components/utis_agflow01.py

Please set the RAGFLOW_URL and RAGFLOW_TOKEN in env variable setting

```
url = f"{RAGFLOW_URL}/api/v1/datasets"  
  
# Build query parameters  
params = {  
    "page": page,  
    "page_size": page_size,  
    "orderby": orderby,  
    "desc": desc  
}  
  
# Add optional parameters if provided  
if name:  
    params["name"] = name  
if dataset_id:  
    params["id"] = dataset_id  
  
# Set up headers with authorization token  
headers = {  
    "Authorization": f"Bearer {RAGFLOW_TOKEN}"  
}  
  
# Make the async HTTP request  
async with httpx.AsyncClient() as client:  
    response = await client.get(url, params=params, headers=headers)  
    response.raise_for_status()  
    data = response.json()["data"]  
  
return data
```

2.3.Starting with an Agent with RAGFlow



6.Assece the RAGFlow in Agent

◆ Example is in

[examples/agent_groupchat/assistant_momery_ragflow.py](#)

The dataset ids in
step 5

```
# Create a factory function to ensure isolated Agent instances for concurrent access.
async def create_agent() -> AssistantAgent:

    # Create model client
    model_client = HepAIChatCompletionClient(
        model="deepseek-ai/deepseek-v3-1",
        base_url="https://aiapi.ihep.ac.cn/apiv2",
        api_key=os.getenv('HEPAI_API_KEY'),
        model_info={
            "vision": False,
            "function_calling": True, # You must sure that the model can handle function calling
            "json_output": True,
            "structured_output": False,
            "family": ModelFamily.GPT_41,
            "multiple_system_messages": True,
        },)

    # Create a RAGFlow memory for your specific knowledge
    RAGFLOW_URL = os.getenv('RAGFLOW_URL')
    RAGFLOW_TOKEN = os.getenv('RAGFLOW_TOKEN')
    ragflow_memory = RAGFlowMemory(
        RAGFlowMemoryConfig(
            RAGFLOW_URL=RAGFLOW_URL,
            RAGFLOW_TOKEN=RAGFLOW_TOKEN,
            dataset_ids=["28e3ad8499b311f0a65d0242ac120006"],
            keyword=True,
        )
    )

    # Create model context
    model_context = TokenLimitedChatCompletionContext(
        model_client=model_client,
        token_limit=100000,
    )

    # Create assistant agent with the model client and memory
    assistant_agent = AssistantAgent(
        name="assistant_agent",
        system_message="你是一个问答助手，需要根据检索到的记忆内容进行回复。",
        description="一个问答助手",
        model_client=model_client,
        model_client_scream=True,
        memory=[ragflow_memory],
        model_context=model_context
    )

    return assistant_agent
```

A Key Feature of OpenDrSai



◆ Development as Application

```
def create_agent() -> AssistantAgent:

    # Define a model client. You can use other model client that implements
    # the `ChatCompletionClient` interface.
    model_client = HepAIChatCompletionClient(
        model="deepseek-ai/deepseek-r1:671b",
        api_key=os.environ.get("HEPAI_API_KEY"),
        base_url="https://aiapi.ihep.ac.cn/apiv2",
    )

    # Define an AssistantAgent with the model, tool, system message, and reflection enabled.
    # The system message instructs the agent via natural language.
    return AssistantAgent(
        name="weather_agent",
        model_client=model_client,
        system_message="You are a helpful assistant.",
        reflect_on_tool_use=False,
        model_client_stream=True, # Enable streaming tokens from the model client.
    )
```

Build
components

```
asyncio.run(
    run_worker(
        # 智能体注册信息
        agent_name="DeepSeek_R1",
        permission='groups: drsai; users: admin, xionddb@ihep.ac.cn',
        description = "DeepSeek_R1 聊天助手.",
        version = "0.1.0",
        logo="https://aiapi.ihep.ac.cn/apiv2/files/file-8572b27d093",
        # 智能体实例
        agent_factory=create_agent,
        # 后端服务配置
        port = 42812,
        no_register=False,
        enable_openwebui_pipeline=True,
        history_mode = "backend",
        # use_api_key_mode = "backend",
    )
)
```

One-button
deployment

```
$ pip install drsai drsai_ui
$ python xx_agent.py # Run Agent
$ drsai ui # Run Interactive UI
```

```
examples > agent_groupchat > assistant_ragflow > assistant_ragflow.py > create_agent

19 from dotenv import load_dotenv
20 load_dotenv()
21
22 RAGFLOW_URL = os.getenv('RAGFLOW_URL')
23 RAGFLOW_TOKEN = os.getenv('RAGFLOW_TOKEN')
24
25
26 # 创建一个工厂函数，用于并发访问时确保后端使用的Agent实例是隔离的。
27 async def create_agent() -> AssistantAgent:
28
29     model_client = HepAIChatCompletionClient(
30         model="deepseek-ai/deepseek-v3-1",
31     )
32
33     ragflow_memory = RagFlowMemory(
34         RagFlowMemoryConfig(
35             RAGFLOW_URL=RAGFLOW_URL,
36             RAGFLOW_TOKEN=RAGFLOW_TOKEN,
37             dataset_ids=["28e3ad8499b311f0a65d0242ac120006"],
38             keyword=True,
39         )
40     )
41
42     # Create assistant agent with ChromaDB memory
43     assistant_agent = AssistantAgent(
44         name="assistant_agent",
45         system_message="你是一个高能物理BESIII实验分析软件BOSS8的问答助手。你需要根据查询到记忆回答用户的相关问题，你的要求如下：
46         1. 严格按照查询到的记忆进行回复，禁止自己编造相关内容；
47         2. 当未查询到相关内容，或者查询的内容与用户的不相关时，请回复“未查询到相关内容”。具体内容参见：https://code.ihep.ac.cn/mr11/boos_docs/-/releas
```

● For more cases, please refer to:
<https://github.com/hepai-lab/drsai/tree/main/examples>



◆ Python Modules in OpenDrSai

◆ drsai.modules.components

- model_client
 - HepAIChatCompletionClient
- tool
 - DrsaiStaticWorkbench
- memory
 - RAGFlowMemory (RAGFlowMemoryConfig)
- task_manager
 - BaseTaskSystem (TaskType, TaskStatus, Task)
- sensor
 - BaseSensor (BaseSensorConfig)

◆ drsai.modules.baseagent

- DrSaiAgent (It can be imported as "from drsai import AssistantAgent or DrSaiAgent")

◆ drsai.modules.groupchat

- AGGroupChat, AGBaseGroupChatManager
- AGRoundRobinGroupChatManager, AGRoundRobinGroupChat
- AGSwarm, AGSwarmGroupChatManager
- AGSelectorGroupChat, AGSelectorGroupChatManager
- RoundRobinGroupChat, RoundRobinGroupChatManager
- DrSaiChatAgentContainer
- DrSaiBaseGroupChat, DrSaiBaseGroupChatManager

◆ drsai.modules.managers

- messages
 - agent_messages (Include AgentEvent and AgentMessage)
 - groupchat_messages (GroupChat message)
- database
 - DatabaseManager, DatabaseManagerConfig
- datamodel
 - UserInput, Thread, SingleTask, Tasks, PlanCheck

03-Build an AI Agent with OpenDrSai

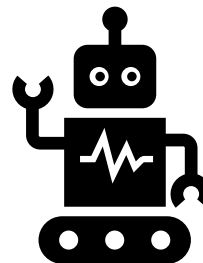
3.1. Building the Components for Agent



Define the agent' s logic and decompose its functions into modular components.

- **ModelClient:** Manages the model' s inputs and outputs
- **ModelContext:** Manages the chat context passed to the ModelClient
- **Memory:** Injects RAG and other contextual information into the ModelContext
- **Workbench:** Manages the invocation of external tools (e.g., functions, MCP)
- **Learning:** Records and learns the agent' s response strategies from the ModelContext based on prompts
- **More as needed**

Orchestrate these components within the agent framework to handle user tasks.



Orchestrate the functions of all components within this function to generate the agent' s final response.

```
async def on_messages_stream(  
    self, messages: Sequence[BaseChatMessage], cancellation_token: CancellationTok  
    ) -> AsyncGenerator[BaseAgentEvent | BaseChatMessage | Response, None]:
```

Building Components in OpenDrSai



```
class ComponentBase(ComponentToConfig[ConfigT], ComponentLoader, Generic[ConfigT]): ...
```

```
class Memory(ABC, ComponentBase[BaseModel]):
```

"""Protocol defining the interface for memory implementations.

A memory is the storage for data that can be used to enrich or modify the model context.

A memory implementation can use any storage mechanism, such as a list, a database, or a file system. It can also use any retrieval mechanism, such as vector search or text search. It is up to the implementation to decide how to store and retrieve data.

1. Inherit "ComponentBase" to implement the basic interfaces of a component

It is a basic memory implementation that is responsible for updating the model context with relevant memory content based on the current model context and querying the memory store.

See :class:`~autogen\_core.memory.ListMemory` for an example implementation.

"""

```
component_type = "memory"
```

```
class RAGFlowMemory(Memory, Component[RAGFlowMemoryConfig]):
```

```
    component_type = "memory"
```

```
    component_provider_override = "drsai.RAGFlowMemory"
```

```
    component_config_schema = RAGFlowMemoryConfig
```

```
    def __init__(
```

```
        self,
```

```
        config: RAGFlowMemoryConfig,
```

```
    ) -> None:
```

```
        self._config = config
```

```
        self._name = config.name or "ragflow_memory"
```

2. Inherit Component to implement the component's specific functionality and parameters

```
def create_ragflow_memory_01(config: RAGFlowMemoryConfig) -> RAGFlowMemory:
```

"""Create a RAGFlowMemory instance with the given configuration.

Args:

config (RAGFlowMemoryConfig): The configuration for RAGFlowMemory.

Returns:

RAGFlowMemory: An instance of RAGFlowMemory.

"""

```
    return RAGFlowMemory(config)
```

```
def dump_ragflow_memory_component(ragflow_memory: RAGFlowMemory) -> Dict[str, Any]:
```

"""Create a RAGFlowMemory instance with the given configuration.

Args:

config (RAGFlowMemoryConfig): The configuration for RAGFlowMemory.

Returns:

RAGFlowMemory: An instance of RAGFlowMemory.

"""

```
    return ragflow_memory.dump_component().model_dump()
```

```
def create_ragflow_memory_from_config(component_config: Dict[str, Any]) -> RAGFlowMemory:
```

"""Create a RAGFlowMemory instance with the given configuration.

Args:

config (RAGFlowMemoryConfig): The configuration for RAGFlowMemory.

Returns:

RAGFlowMemory: An instance of RAGFlowMemory.

"""

```
    return ComponentLoader.load_component(component_config)
```

Components for Model Client



◆ Methods for ChatCompletionClient:

- create/create_stream
- close
- total_usage/actual_usage
- count_tokens
- remaining_tokens
- model_info

◆ “model_info” is very important!!!

```
class HepAIChatCompletionClient(OpenAIChatCompletionClient, Component[HepAIClientConfigurationConfigModel]):  
  
    component_type = "model"  
    component_config_schema = HepAIClientConfigurationConfigModel  
    component_provider_override = "drsai.HepAIChatCompletionClient"  
  
    def __init__(self, **kwargs: Unpack[HepAIClientConfiguration]):  
  
        if "api_key" not in kwargs:  
            kwargs["api_key"] = os.environ.get("HEPAI_API_KEY")  
        if "base_url" not in kwargs:  
            kwargs["base_url"] = "https://aiapi.ihep.ac.cn/apiv2"  
  
        if "model_info" not in kwargs:  
            model_info={  
                "vision": False,  
                "function_calling": False, # You must sure that the model can handle function calling  
                "json_output": False,  
                "structured_output": False,  
                "family": ModelFamily.UNKNOWN,  
                "multiple_system_messages": False,  
            }  
            kwargs["model_info"] = model_info
```

Components for Model Context



◆ Methods for ChatCompletionContext:

- add_message
- get_messages
- save_state
- load_state

◆ Short/Long-Term Memory

◆ RAG/External Knowledge

```
class UnboundedChatCompletionContext(ChatCompletionContext, Component[UnboundedChatCompletionContextConfig]):  
    """An unbounded chat completion context that keeps a view of the all the messages."""  
  
    component_config_schema = UnboundedChatCompletionContextConfig  
    component_provider_override = "autogen_core.model_context.UnboundedChatCompletionContext"  
  
    async def get_messages(self) -> List[LLMMessage]:  
        """Get at most `buffer_size` recent messages."""  
        return self._messages  
  
    def _to_config(self) -> UnboundedChatCompletionContextConfig:  
        return UnboundedChatCompletionContextConfig(initial_messages=self._initial_messages)  
  
    @classmethod  
    def _from_config(cls, config: UnboundedChatCompletionContextConfig) -> Self:  
        return cls(initial_messages=config.initial_messages)
```

Components for Workbench(Tools)



◆ Methods for Workbench:

- list_tools
- call_tool
- start
- stop
- reset
- save_state
- load_state
- __aenter__/_aexit__

```
class StaticWorkbench(Workbench, Component[StaticWorkbenchConfig]):  
    """  
    A workbench that provides a static set of tools that do not change after  
    each tool execution.  
  
    Args:  
        tools (List[BaseTool[Any, Any]]): A list of tools to be included in the workbench.  
        The tools should be subclasses of :class:`~autogen_core.tools.BaseTool`.  
    """  
  
    component_provider_override = "autogen_core.tools.StaticWorkbench"  
    component_config_schema = StaticWorkbenchConfig  
  
    def __init__(self, tools: List[BaseTool[Any, Any]]) -> None:  
        self._tools = tools
```

```
class BaseTool(ABC, Tool, Generic[ArgsT, ReturnT], ComponentBase[BaseModel]):  
    component_type = "tool"  
  
    def __init__(  
        self,  
        args_type: Type[ArgsT],  
        return_type: Type[ReturnT],  
        name: str,  
        description: str,  
        strict: bool = False,  
    ) -> None:  
        self._args_type = args_type  
        # Normalize Annotated to the base type.  
        self._return_type = normalize_annotated_type(return_type)  
        self._name = name  
        self._description = description  
        self._strict = strict
```

```
class FunctionTool(BaseTool[BaseModel, BaseModel], Component[FunctionToolConfig]):
```

In Agent:

```
# STEP 4B: Execute tool calls  
executed_calls_and_results = await asyncio.gather(  
    *  
    cls._execute_tool_call(  
        tool_call=call,  
        workbench=workbench,  
        handoff_tools=handoff_tools,  
        agent_name=agent_name,  
        cancellation_token=cancellation_token,  
    )  
    for call in model_result.content  
)  
exec_results = [result for _, result in executed_calls_and_results]
```

```
# Handle normal tool call using workbench.  
result = await workbench.call_tool(  
    name=tool_call.name,  
    arguments=arguments,  
    cancellation_token=cancellation_token,  
)  
return (  
    tool_call,  
    FunctionExecutionResult(  
        content=result.to_text(),  
        call_id=tool_call.id,  
        is_error=result.is_error,  
        name=tool_call.name,  
    )),
```

Components for Memory and Learning



◆ Methods for Memory:

- update_context
- query
- add
- clear
- close



- **query** -> RAG/Graph Knowledge
- **update_context**-> add to model_context

In Agent:

```
events: List[MemoryQueryEvent] = []
if memory:
    for mem in memory:
        update_context_result = await mem.update_context(model_context)
        if update_context_result and len(update_context_result.memories.results) > 0:
            memory_query_event_msg = MemoryQueryEvent(
                content=update_context_result.memories.results,
                source=agent_name,
            )
            events.append(memory_query_event_msg)
return events
```

◆ Methods for Learning:

- learn_from_context
- update_context
- clear
- close



```
class BaseLearning(ABC, ComponentBase[BaseModel]):
    component_type = "learning"

    @abstractmethod
    async def learn_from_context(
        self,
        model_context: ChatCompletionContext,
        learning_config: BaseLearningConfig,
        cancellation_token: CancellationToken | None = None,
        **kwargs: Any,
    ) -> Any:
        """
        Retrieve data from the sensor.

        Args:
            serson_config: The configuration for the sensor.
            cancellation_token: A cancellation token to cancel the operation.

        Returns:
            Any: The retrieved data.
        """
```

Why we need components?



Concepts of component: Reusable and composable minimal functional units

- ◆ **Advantages of components:** The core of a component-based architecture lies in modularizing, standardizing, and reusing the functional elements of AI agents
 - ✓ **Standardization and Functional Reuse:** Significantly reduces AI agent development costs through functional reuse and standardized interfaces
 - ✓ **Enhanced Internal Collaboration Efficiency:** Enables efficient collaboration among multiple agents via standardized communication protocols and task decomposition mechanisms
 - ✓ ...

3.2. Building the Management System for Agent



You may need to manage the agent' s configuration, runtime state, and other related information.

- **State Management System:** Controls the agent' s runtime states such as pause, restart, and state reset
- **Agent Configuration Management System:** Stores the agent' s global variables and configuration states
- More as needed



◆ Methods for State Management in DrSaiAgent:

- lazy_init
- close
- pause
- resume
- save_state/load_state
- on_reset
- ...

```
async def save_state(self) -> Mapping[str, Any]:
    """Save the current state of the assistant agent."""
    model_context_state = await self._model_context.save_state()
    return AssistantAgentState(llm_context=model_context_state).model_dump()

async def load_state(self, state: Mapping[str, Any]) -> None:
    """Load the state of the assistant agent"""
    assistant_agent_state = AssistantAgentState.model_validate(state)
    # Load the model context state.
    await self._model_context.load_state(assistant_agent_state.llm_context)
```

```
async def lazy_init(self, cancellation_token: CancellationToken) -> None:
    """Initialize the tools and models needed by the agent."""
    pass

async def close(self, cancellation_token: CancellationToken) -> None:
    """Clean up resources used by the agent.

    This method:
    | ...
    | """

    logger.info(f"Closing {self.name}...")
    if self._cancellation_token is not None and not self._cancellation_token.is_cancelled():
        self._cancellation_token.cancel()
    # Close the model client.
    await self._model_client.close()

async def pause(self, cancellation_token: CancellationToken) -> None:
    """Pause the agent by setting the paused state."""
    logger.info(f"Pausing {self.name}...")
    if self._cancellation_token is not None and not self._cancellation_token.is_cancelled():
        self._cancellation_token.cancel()
    self.is_paused = True
    self._paused.set()

async def resume(self, cancellation_token: CancellationToken) -> None:
    """Resume the agent by clearing the paused state."""
    self.is_paused = False
    self._paused.clear()
```

State Management System



◆ Variables for State Management:

- self._paused
- cancellation_token
- ...

● “cancellation_token” in Agent methods :

- ChatCompletionClient component
- Workbench component
- RAGFlowMemory
- Sensor

◆ “self._paused” in Agent methods :

- on_messages_stream

```
async def pause(self, cancellation_token: CancellationToken) -> None:
    """Pause the agent by setting the paused state."""
    logger.info(f"Pausing {self.name}...")
    if self._cancellation_token is not None and not self._cancellation_token.is_cancelled():
        self._cancellation_token.cancel()
    self.is_paused = True
    self._paused.set()
```

cancellation_token.cancel()

is_paused = True

```
)
if cancellation_token is not None:
    cancellation_token.link_future(stream_future)
stream = await stream_future
while True:
    try:
        chunk_future = asyncio.ensure_future(anext(stream))
        if cancellation_token is not None:
            cancellation_token.link_future(chunk_future)
        chunk = await chunk_future
        yield chunk
    except StopAsyncIteration:
        break
```

```
# STEP 3: Run the first inference
model_result = None
async for inference_output in self._call_llm(
    model_client=model_client,
    model_client_stream=model_client_stream,
    system_messages=system_messages,
    model_context=model_context,
    workbench=workbench,
    handoff_tools=handoff_tools,
    agent_name=agent_name,
    cancellation_token=cancellation_token,
    output_content_type=output_content_type,
):
    if self.is_paused:
        raise asyncio.CancelledError()

    if isinstance(inference_output, CreateResult):
        model_result = inference_output
```

Agent Configuration Management System



◆ Components in Agent:

➤ AssistantAgentConfig

```
class AssistantAgentConfig(BaseModel):
    """The declarative configuration for the assistant agent."""

    name: str
    model_client: ComponentModel
    tools: List[ComponentModel] | None = None
    workbench: ComponentModel | None = None
    handoffs: List[HandoffBase | str] | None = None
    model_context: ComponentModel | None = None
    memory: List[ComponentModel] | None = None
    description: str
    system_message: str | None = None
    model_client_stream: bool = False
    reflect_on_tool_use: bool
    tool_call_summary_format: str
    metadata: Dict[str, str] | None = None
    structured_message_factory: ComponentModel | None = None
```

```
def _to_config(self) -> AssistantAgentConfig:
    """Convert the assistant agent to a declarative config."""

    return AssistantAgentConfig(
        name=self.name,
        model_client=self._model_client.dump_component(),
        tools=None, # versionchanged:: v0.5.5 Now tools are not serialized, Cause they are p
        workbench=self._workbench.dump_component() if self._workbench else None,
        handoffs=list(self._handoffs.values()) if self._handoffs else None,
        model_context=self._model_context.dump_component(),
        memory=[memory.dump_component() for memory in self._memory] if self._memory else None,
        description=self.description,
        system_message=self._system_messages[0].content
        if self._system_messages and isinstance(self._system_messages[0].content, str)
        else None,
        model_client_stream=self._model_client_stream,
        reflect_on_tool_use=self._reflect_on_tool_use,
        tool_call_summary_format=self._tool_call_summary_format,
        structured_message_factory=self._structured_message_factory.dump_component()
        if self._structured_message_factory
        else None,
        metadata=self._metadata,
    )
```

Agent Configuration Management System



◆ Resume from configs and states:

```
async def save_state(self) -> Mapping[str, Any]:
    """Save the current state of the assistant agent."""
    model_context_state = await self._model_context.save_state()
    return AssistantAgentState(llm_context=model_context_state).model_dump()

async def load_state(self, state: Mapping[str, Any]) -> None:
    """Load the state of the assistant agent"""
    assistant_agent_state = AssistantAgentState.model_validate(state)
    # Load the model context state.
    await self._model_context.load_state(assistant_agent_state.llm_context)
```



```
@classmethod
def _from_config(
    cls, config: AssistantAgentConfig,
    db_manager: DatabaseManager,
    memory_function: Callable = None,
    reply_function: Callable = None,
    **kwargs,
) -> Self:
    """Create an assistant agent from a declarative config."""
    if config.structured_message_factory:
        structured_message_factory = StructuredMessageFactory.load_component(config.structured_message_factory)
        format_string = structured_message_factory.format_string
        output_content_type = structured_message_factory.ContentModel
    else:
        format_string = None
        output_content_type = None

    return cls(
        name=config.name,
        model_client=ChatCompletionClient.load_component(config.model_client),
        workbench=Workbench.load_component(config.workbench) if config.workbench else None,
        handoffs=config.handoffs,
        model_context=ChatCompletionContext.load_component(config.model_context) if config.model_context else None,
        tools=[BaseTool.load_component(tool) for tool in config.tools] if config.tools else None,
        memory=[Memory.load_component(memory) for memory in config.memory] if config.memory else None,
        description=config.description,
        system_message=config.system_message,
        model_client_stream=config.model_client_stream,
        reflect_on_tool_use=config.reflect_on_tool_use,
        tool_call_summary_format=config.tool_call_summary_format,
        output_content_type=output_content_type,
        output_content_type_format=format_string,
        metadata=config.metadata,
        memory_function=memory_function,
        reply_function=reply_function,
        db_manager=db_manager,
        **kwargs,
    )
```

Arguments and Database Manager for Applications



- **db_manager**: Class for backend data management of Agent APP
- **thread_id**: chat_id from frontend
- **user_id**: User's email or other info label

```
db_manager: DatabaseManager = None,  
thread_id: str = None,  
user_id: str = None,  
**kwargs,  
|   |:  
...  
  
memory_function: 自定义的memory_function, 用于RAG检索等功能, 为大模型回复增加最新的知识  
reply_function: 自定义的reply_function, 用于自定义对话回复的定制  
db_manager: 数据库管理器  
thread_id: 前端当前会话的id  
user_id: 用户id  
...
```

➤ **Methods in db_manager:**

- get
- upsert
- delete
- reset_db
- close

➤ **Default data model in db_manager:**

- **UserInput**: Update user's input info at each accessing
- **Thread**: Update user's task and output messages (use add)

Default database file in ~/.drsai/drsai.db

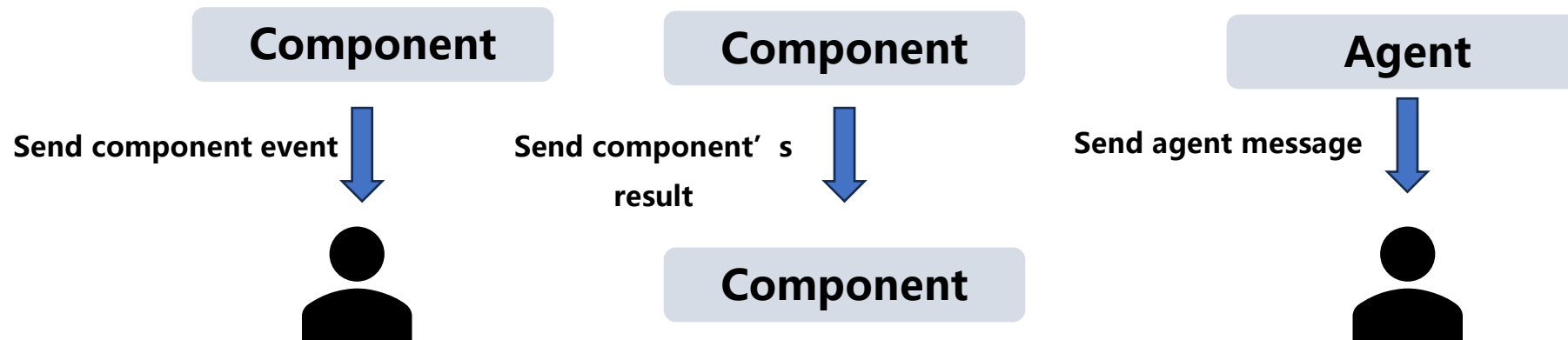
3.3. Why do we need events and notifications?



How can we orchestrate different components and modules efficiently? -> Events and Notifications

◆ Advantages of events and notifications:

- ✓ **Standardized Module Communication:** Enables accurate information exchange between components within an agent and across multiple agents using a unified message format.
- ✓ **Output Visualization:** Displays messages from different modules in separate views, making the internal workflow of the agent transparent.
- ✓ ...

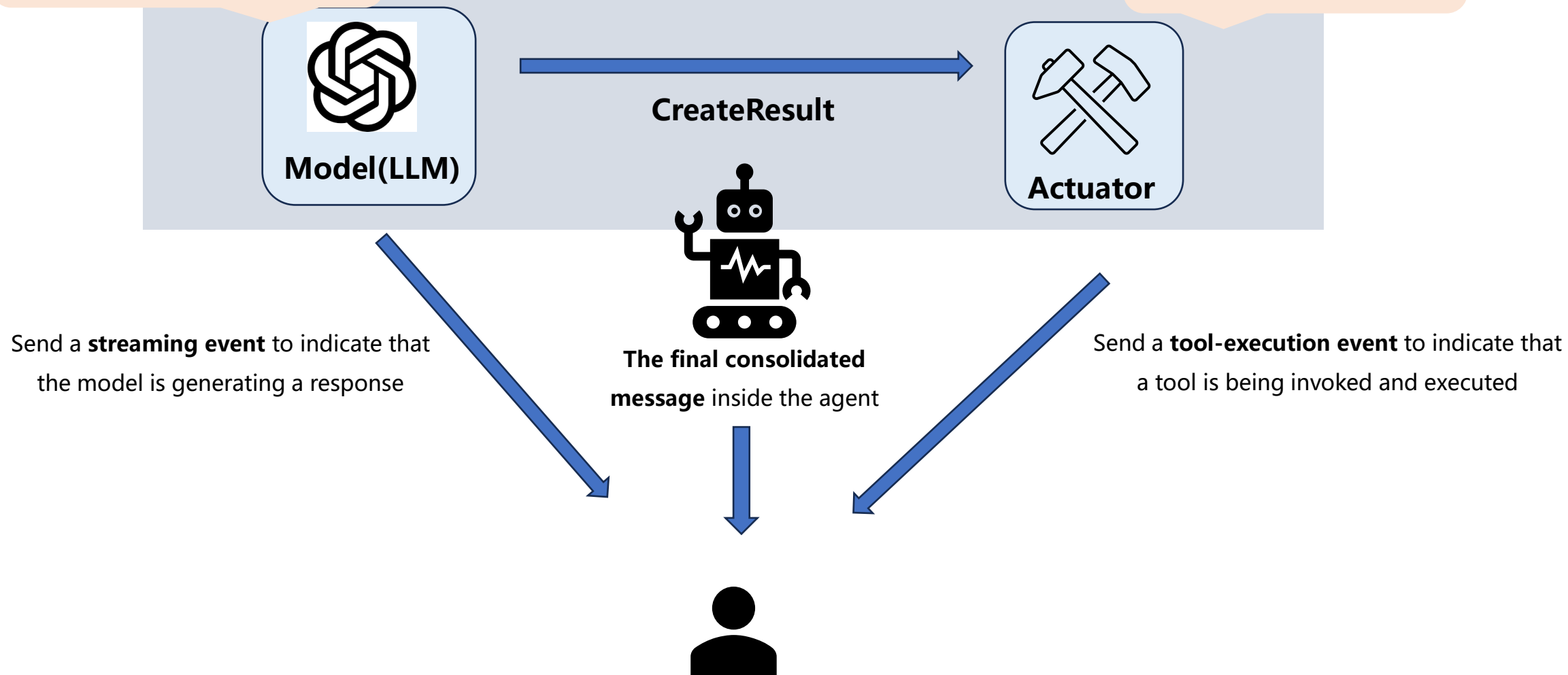


How the Event and Notification Mechanism Works



This is the result I generated based on the user's task. Check the CreateResult to see whether a tool needs to be invoked.

I will review the CreateResult you provided and decide whether a tool needs to be called.



Event and Message Type



Streaming Chunks:

◆ BaseAgentEvent:

- ToolCallRequestEvent
- CodeGenerationEvent
- CodeExecutionEvent
- ToolCallExecutionEvent
- UserInputRequestedEvent
- MemoryQueryEvent
- ModelClientStreamingChunkEvent
- ThoughtEvent
- SelectSpeakerEvent
- SelectorEvent
- TaskEvent
- AgentLogEvent
- ToolLongTaskEvent

Agent Message: ◆ BaseChatMessage:

- TextMessage
- StopMessage
- HandoffMessage
- ToolCallSummaryMessage
- StructuredMessage
- MultiModalMessage
- AgentLongTaskMessage
- LongTaskQueryMessage

```
class BaseAgentEvent(BaseMessage, ABC):
    """Base class for agent events.

    .. note::
        If you want to create a new message type for signaling observable even
        to user and application, inherit from this class.

    Agent events are used to signal actions and thoughts produced by agents
    and teams to user and applications. They are not used for agent-to-agent
    communication and are not expected to be processed by other agents.

    You should override the :meth:`to_text` method if you want to provide
    a custom rendering of the content.
    """

    source: str
    """The name of the agent that sent this message."""

    models_usage: RequestUsage | None = None
    """The model client usage incurred when producing this message."""

    metadata: Dict[str, str] = {}
    """Additional metadata about the message."""
```

```
class BaseChatMessage(BaseMessage, ABC):
    """Abstract base class for chat messages.

    .. note::
        If you want to create a new message type that is used for agent-to-agent
        communication, inherit from this class, or simply use
        :class:`StructuredMessage` if your content type is a subclass of
        Pydantic BaseModel.

    This class is used for messages that are sent between agents in a chat
    conversation. Agents are expected to process the content of the
    message using models and return a response as another :class:`BaseChatMessage`.
    """

    source: str
    """The name of the agent that sent this message."""

    models_usage: RequestUsage | None = None
    """The model client usage incurred when producing this message."""

    metadata: Dict[str, str] = {}
    """Additional metadata about the message."""

    @abstractmethod
    def to_model_text(self) -> str:
        """Convert the content of the message to text-only representation.
        This is used for creating text-only content for models.

        This is not used for rendering the message in console. For that, use
        :meth:`~BaseMessage.to_text`.

        The difference between this and :meth:`to_model_message` is that this
        is used to construct parts of the a message for the model client,
        while :meth:`to_model_message` is used to create a complete message
        for the model client.
        """
        ...

    @abstractmethod
    def to_model_message(self) -> UserMessage:
        """Convert the message content to a :class:`~autogen_core.models.UserMessage`
        for use with model client, e.g., :class:`~autogen_core.models.ChatCompletionClient`.
        """
```

Event and Message Type



Access frontend by agent' s event:

- ModelClientStreamingChunkEvent
- AgentLogEvent
- TaskEvent

Example is in

examples/agent_groupchat/assistant_custom_log_event-
interaction.py

Task_Assistant

BOSS8是BESIII离线软件系统（BESIII Offline Software System）的第八个主要版本，用于高能物理实验BESIII（北京谱仪III）的数据处理、模拟和物理分析。它基于C++和Python开发，提供了完整的事件重建、蒙特卡罗模拟以及物理分析工具链。

具体内容参见：https://code.ihep.ac.cn/mrli/boss_docs/-/releases

你好！请问有什么关于BOSS8的问题需要我帮助解答吗？

什么是BOSS8

hi

什么是bosds8

TaskManager

LogExecution

Terminal

LogExecution

Terminal

parent_task for test

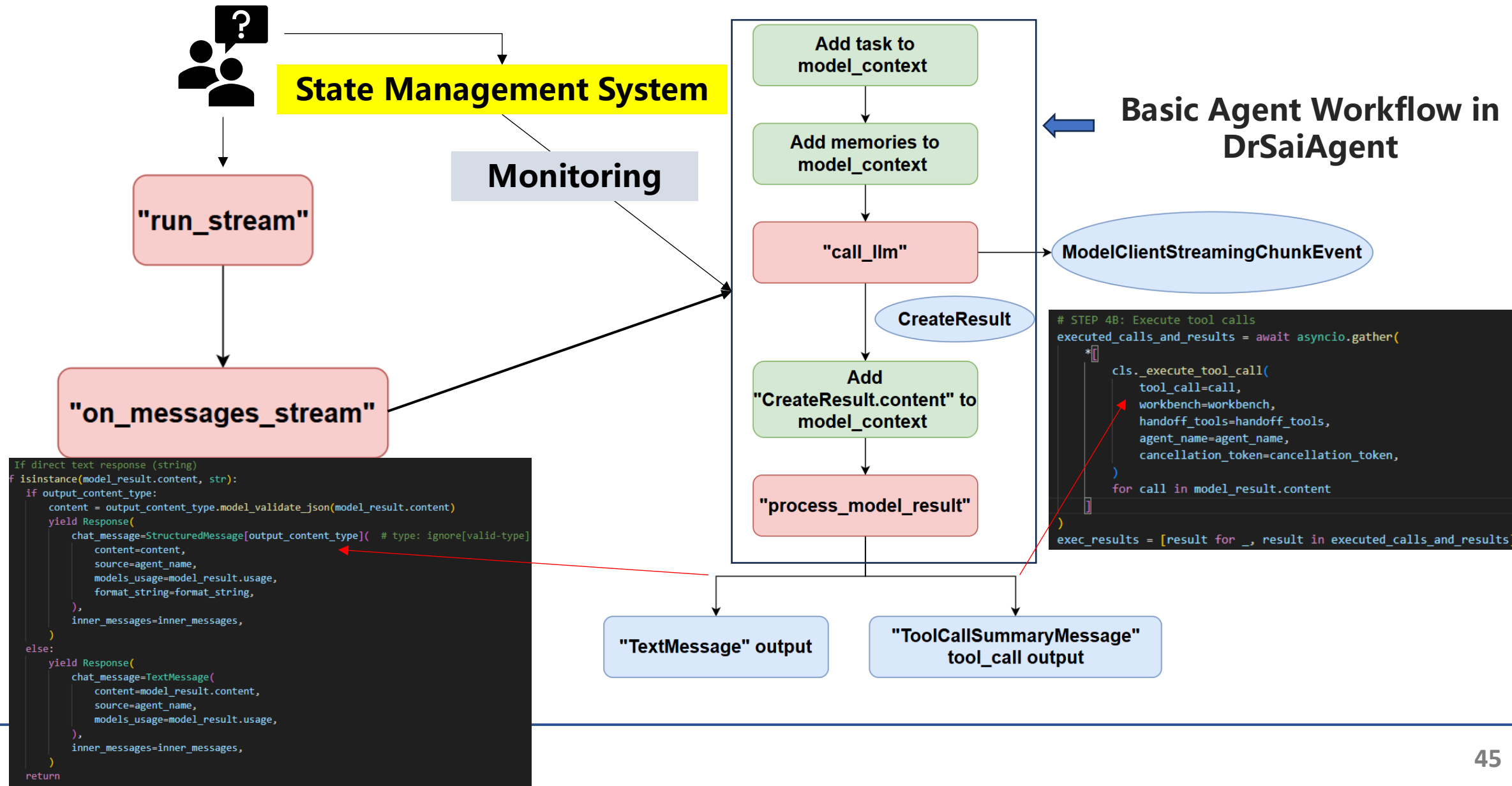
sub_task_1 for test

sub_task_2 for test

sub_task_3 for test

memory to the model contextAdding the memory to the model
oryContent(content='根据您提供的段落内容，总结如下：\n\n开发
路径中的后缀`-opt`替换为`-dbg`来使用 BOSS8 的调试版本。此
`boss-build -d`命令构建本地仓库的调试版本并运行，方法与调试
。', mime_type=<MemoryMimeType.TEXT: 'text/plain'>,
content': '根据您提供的段落内容，总结如下：\n\n开发者可以通过
缀`-opt`替换为`-dbg`来使用 BOSS8 的调试版本。此外，可以使

3.4. Building the AI Agent



Orchestrate the functions of all components within this function to generate the agent's final response.

```
✦ async def on_messages_stream(  
    self, messages: Sequence[BaseChatMessage], cancellation_token: CancellationToken  
    ) -> AsyncGenerator[BaseAgentEvent | BaseChatMessage | Response, None]:
```



This function serves as the external interface — it receives external tasks and calls `on_messages_stream` to produce responses.

```
async def run_stream(  
    self,  
    *,  
    task: str | BaseChatMessage | Sequence[BaseChatMessage] | None = None,  
    cancellation_token: CancellationToken | None = None,  
    ) -> AsyncGenerator[BaseAgentEvent | BaseChatMessage | TaskResult, None]:
```

```
@dataclass(kw_only=True)  
class Response:  
    ✦ ✦ """A response from calling :meth:`ChatAgent.on_messages`."""  
  
    chat_message: BaseChatMessage  
    """A chat message produced by the agent as the response."""  
  
    inner_messages: Sequence[BaseAgentEvent | BaseChatMessage] | None = None  
    """Inner messages produced by the agent, they can be :class:`BaseAgentEvent`  
    or :class:`BaseChatMessage`."""
```

```
✦ class TaskResult(BaseModel):  
    ✦ """Result of running a task."""  
  
    messages: Sequence[BaseAgentEvent | BaseChatMessage]  
    """Messages produced by the task."""  
  
    stop_reason: str | None = None  
    """The reason the task stopped."""
```



- ◆ The complete AI Agent framework code is located at:
`python/packages/drsai/src/drsai/modules/baseagent/drsaiagent.py`

- ◆ Components examples in `examples/components`:
 - `examples/components/model_client01.py`
 - `examples/components/memory_ragflow01.py`
 - `examples/components/tools01.py`

- ◆ Agent examples in `examples/agent_groupchat`:
 - MCP: `examples/tools/MCP_Long_task/long_task_test.py`
 - `examples/agent_groupchat/groupchat_task_LongTask/groupchat_task/Long_MCP_Agent.py`

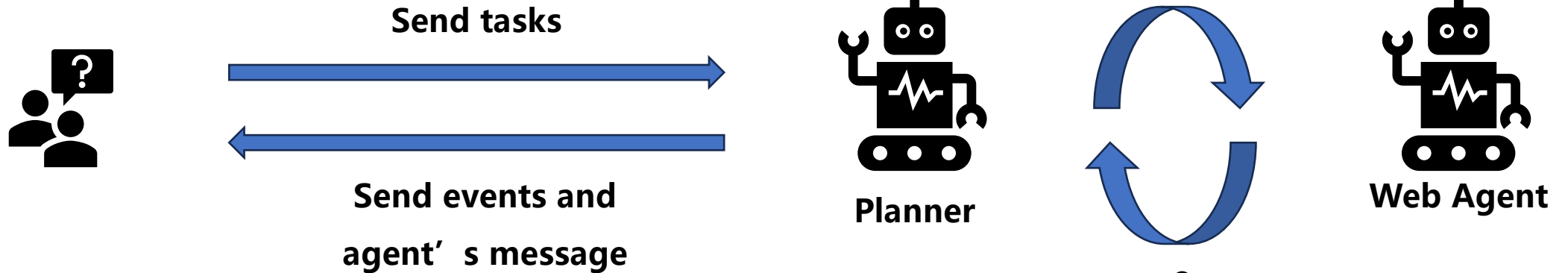
04-Multi-Agent System(MAS)

4.1. Concept of Multi-Agent System(MAS)



◆ Necessary Modules/Subsystem:

- **1.Multi-Agent Collaboration System:** Defines how multiple agents cooperate, such as task-based scheduling, workflow-based scheduling, or function-based scheduling.
- **2.Management System for Participants:** Manages agent information registration and scheduling.



- **3. State Management:** Manages the runtime state of the multi-agent system, including task interruption or completion.
- **4. System-Level Memory (and Learning Mechanisms):** Stores and propagates contextual information generated during multi-agent system operations.

4.2.Starting with an RoundRobin GroupChat



- ◆ **OpenDrSai provides three orchestration logics for multi-agent systems:**
 - **RoundRobin:** Selects agents in a fixed sequence to respond to tasks.
 - Located in: `examples/agent_groupchat/groupchat_roundgroup.py`
 - **Selector:** Uses an LLM to choose the appropriate agent based on the task and chat context.
 - Located in: `examples/agent_groupchat/groupchat_select.py`
 - **Swarm:** Uses the Handoff mechanism and Function Call to select the appropriate agent to respond.
 - Located in: `examples/agent_groupchat/groupchat_swarm.py`

Starting with an RoundRobin GroupChat



```
# Create a factory function to ensure isolated Agent instances for concurrent access.
def create_team() -> RoundRobinGroupChat:
    # Create an OpenAI model client.
    model_client = HepAIChatCompletionClient(
        model="openai/gpt-4o",
    )

    # Create the primary agent.
    primary_agent = DrSaiAgent(
        "primary",
        model_client=model_client,
        system_message="You are a helpful AI assistant.",
        model_client_stream=True,
    )

    # Create the critic agent.
    critic_agent = DrSaiAgent(
        "critic",
        model_client=model_client,
        system_message="Provide constructive feedback. Respond with 'APPROVE' to when your feedback is approved.",
        model_client_stream=True,
    )

    # Define a termination condition that stops the task if the critic approves.
    text_termination = TextMentionTermination("APPROVE")

    # Create a team with the primary and critic agents.
    return RoundRobinGroupChat(
        participants=[primary_agent, critic_agent],
        termination_condition=text_termination
    )
```

- Located in:
examples/agent_groupchat/groupchat_roundgroup.py

1. Create the participants

4. Wrap the GroupChat inside a function
5. Register as an API server
6. Start a UI for Agent

2. Create the TerminationCondition

3. Create the RoundRobin GroupChat

4.3. Building Your own Multi-Agent System(MAS)



How to build your own multi-agent system within OpenDrSai?

- Three main parts need to be customized.

DrSaiBaseGroupChat

- Prepare the parameters passed into **DrSaiBaseGroupChatManager** and your Agents from the entry interface, then start the DrSaiBaseGroupChatManager to begin the GroupChat.
 - **DrSaiBaseGroupChat** is located in: `examples/agent_groupchat/groupchat_roundgroup.py`
 - **AGRoundRobinGroupChat** is located in: `python/packages/drsai/src/drsai/modules/groupchat/ag_round_robin_group_chat.py`

DrSaiBaseGroupChatManager

- Orchestrate the operational logic of the multi-agent system, including essential modules and subsystems such as the Multi-Agent Collaboration System, Participant Management System, State Management, and System-Level Memory required for running a multi-agent system.
 - **DrSaiBaseGroupChat** is located in: `examples/agent_groupchat/groupchat_roundgroup.py`
 - **AGRoundRobinGroupChatManager** is located in: `python/packages/drsai/src/drsai/modules/groupchat/drsai_base_group_chat_manager.py`

Your Agents

- Build your own agents based on Part 2 and 3 and wrap them with the **DrSaiChatAgentContainer** type to ensure they can be accessed by the **DrSaiBaseGroupChatManager**.
 - **DrSaiChatAgentContainer** is located in: `python/packages/drsai/src/drsai/modules/groupchat/drsai_base_agent_container.py`

Communication mechanism in MAS



Core Concept in AutoGen and OpenDrSai: Using **GroupChat events** to mark and handle different states of the Multi-Agent System.

Based on Events and Notifications

➤ GroupChat Events:

- GroupChatLazyInit
- GroupChatStart
- GroupChatAgentResponse
- GroupChatAgentLongTask
- GroupChatMessage
- GroupChatRequestPublish
- GroupChatClose
- GroupChatReset
- GroupChatError
- GroupChatPause
- GroupChatResume
- GroupChatTermination



DrSaiBaseGroupChatManager

Handle and send/emit various GroupChat events in the **DrSaiBaseGroupChatManager** framework

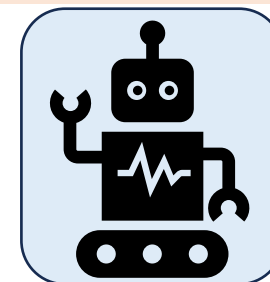
GroupChat events



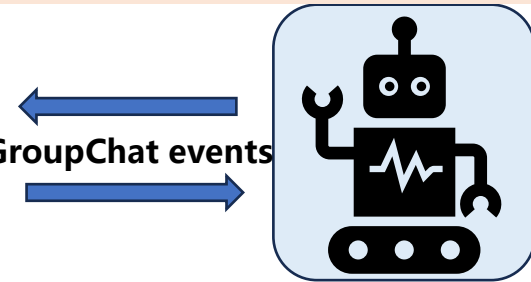
GroupChat events



Invoking the Agent instance within the **DrSaiChatAgentContainer** framework to handle and send/emit various GroupChat events



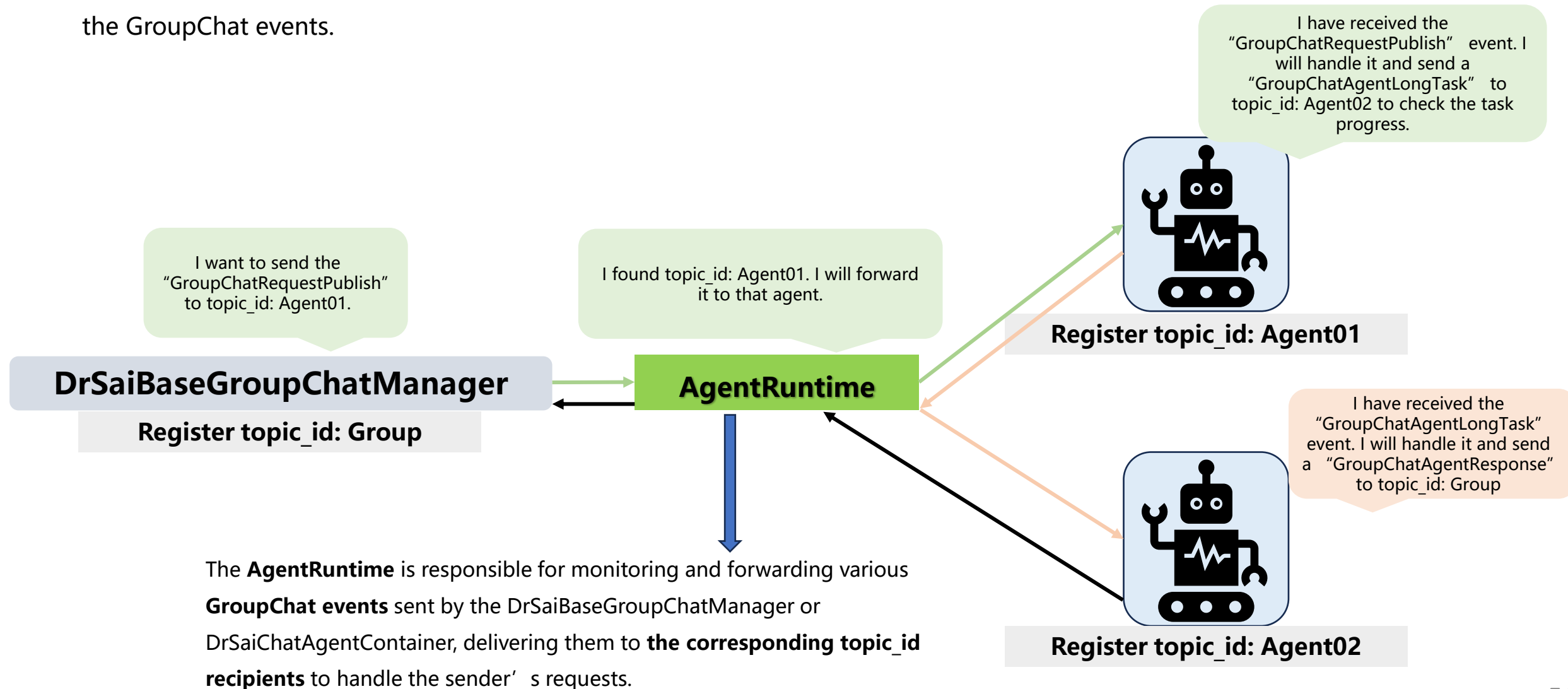
GroupChat events



Communication mechanism in MAS



Core Concept in AutoGen and OpenDrSai: Use the **Topic and Subscription mechanism** to determine who should receive the GroupChat events.



Building Your own Multi-Agent System



◆ Step1: Building your own multi-agent collaboration system within DrSaiBaseGroupChatManager

```
class AGRoundRobinGroupChatManager(DrSaiBaseGroupChatManager):
    """A group chat manager that selects the next speaker in a round-robin fashion."""

    def __init__(
        self,
        name: str,
        group_topic_type: str,
        output_topic_type: str,
        participant_topic_types: List[str],
        participant_names: List[str],
        participant_descriptions: List[str],
        output_message_queue: asyncio.Queue[BaseAgentEvent | BaseChatMessage | GroupChatTermination],
        termination_condition: TerminationCondition | None,
        max_turns: int | None,
        message_factory: MessageFactory,
        emit_team_events: bool,
        db_manager: DatabaseManager = None,
        **kwargs: Any
    ) -> None:
        super().__init__(
            name=name,
            group_topic_type=group_topic_type,
            output_topic_type=output_topic_type,
            participant_topic_types=participant_topic_types,
            participant_names=participant_names,
            participant_descriptions=participant_descriptions,
            output_message_queue=output_message_queue,
            termination_condition=termination_condition,
            max_turns=max_turns,
            message_factory=message_factory,
            emit_team_events=emit_team_events,
            db_manager=db_manager,
            **kwargs
        )
        self._next_speaker_index = 0
```

Inherit "DrSaiBaseGroupChatManager" to implement your own collaboration system

Basic Functions

```
@abstractmethod
async def select_speaker(
    self, thread: List[BaseAgentEvent | BaseChatMessage]
) -> str:
    """Select a speaker from the participants in a round-robin fashion."""
    ...

@abstractmethod
async def save_state(self) -> Mapping[str, Any]:
    """save the state of the group chat manager."""
    ...

@abstractmethod
async def load_state(self, state: Mapping[str, Any]) -> None:
    """Load the state of the group chat manager."""
    ...

@abstractmethod
async def pause(self) -> None:
    """Pause the group chat manager."""
    ...

@abstractmethod
async def resume(self) -> None:
    """Resume the group chat manager."""
    ...

@abstractmethod
async def close(self) -> None:
    """Close any resources."""
    ...

@abstractmethod
async def reset(self) -> None:
    """Reset the group chat manager."""
    ...
```

For collaboration

For state management

Building Your own Multi-Agent System



◆ Step1: Building your own multi-agent collaboration system within DrSaiBaseGroupChatManager

State management



```
async def reset(self) -> None:
    self._current_turn = 0
    self._message_thread.clear()
    if self._termination_condition is not None:
        await self._termination_condition.reset()
    self._next_speaker_index = 0

async def save_state(self) -> Mapping[str, Any]:
    state = RoundRobinManagerState(
        message_thread=list(self._message_thread),
        current_turn=self._current_turn,
        next_speaker_index=self._next_speaker_index,
    )
    return state.model_dump()

async def load_state(self, state: Mapping[str, Any]) -> None:
    round_robin_state = RoundRobinManagerState.model_validate(state)
    self._message_thread = list(round_robin_state.message_thread)
    self._current_turn = round_robin_state.current_turn
    self._next_speaker_index = round_robin_state.next_speaker_index
```

Example of Round Robin mechanism:



```
async def select_speaker(self, thread: List[AgentEvent | ChatMessage]) -> str:
    """Select a speaker from the participants in a round-robin fashion."""
    current_speaker_index = self._next_speaker_index
    self._next_speaker_index = (current_speaker_index + 1) % len(self._participant_names)
    current_speaker = self._participant_names[current_speaker_index]
    return current_speaker
```


Building Your own Multi-Agent System



◆ Step1: Building your own multi-agent collaboration system within DrSaiBaseGroupChatManager

Moreover, you can develop these functions about handling and sending/emitting various GroupChat events in the DrSaiBaseGroupChatManager

```
@rpc
async def handle_lazy_init(self, message: GroupChatLazyInit, ctx: MessageContext) -> None:
    """Handle the lazy_init for a group chat."""
    pass

@rpc
async def handle_start(self, message: GroupChatStart, ctx: MessageContext) -> None:
    """Handle the start of a group chat by selecting a speaker to start the conversation."""

@event
async def handle_agent_response(self, message: GroupChatAgentResponse, ctx: MessageContext) -> None:
    try:

@event
async def handle_long_task(self, message: GroupChatAgentLongTask, ctx: MessageContext) -> None:
    """Handle Agent's long task."""

@event
async def handle_group_chat_message(self, message: GroupChatMessage, ctx: MessageContext) -> None:
    """Handle a group chat message by appending the content to its output message queue."""
    await self._output_message_queue.put(message.message)

@event
async def handle_group_chat_error(self, message: GroupChatError, ctx: MessageContext) -> None:
    """Handle a group chat error by logging the error and signaling termination."""
    await self._signal_termination_with_error(message.error)
```

- **GroupChatLazyInit:** Handles initialization events required when agents and the multi-agent system start
- **GroupChatStart:** Handles the initial conversation of the multi-agent system
- **GroupChatAgentResponse:** Handles agent responses
- **GroupChatAgentLongTask:** Handles long-task execution notifications issued by agents
- **GroupChatMessage:** Handles messages emitted by the multi-agent system
- **GroupChatError:** Handles errors that occur within the multi-agent system

Building Your own Multi-Agent System



◆ Step1: Building your own multi-agent collaboration system within DrSaiBaseGroupChatManager

Use “**send_message**” in DrSaiBaseGroupChatManager or DrSaiChatAgentContainer to send the corresponding GroupChat event to the function decorated with @rpc that handles that event. “**send_message**” is used for point-to-point delivery of GroupChat events between the GroupChatManager and ChatAgentContainer, with **the recipient** specifying the target.

```
await self._runtime.send_message(  
    GroupChatStart(messages=messages),  
    recipient=AgentId(type=self._group_chat_manager_topic_type, key=self._team_id),  
    cancellation_token=cancellation_token,  
)
```



```
@rpc  
async def handle_start(self, message: GroupChatStart, ctx: MessageContext) -> None:  
    """Handle the start of a group chat by selecting a speaker to start the conversation."""
```

Use “**publish_message**” in DrSaiBaseGroupChatManager or DrSaiChatAgentContainer to publish the corresponding GroupChat event to the function decorated with @event that handles that event. “**publish_message**” is used for broadcasting GroupChat events from the GroupChatManager or ChatAgentContainer. Events are broadcast to every GroupChatManager or ChatAgentContainer subscribed to **the specified topic_id**.

```
# send request to next speaker  
await self.publish_message(  
    GroupChatRequestPublish(),  
    topic_id=DefaultTopicId(  
        type=self._participant_name_to_topic_type[next_speaker]  
    ),  
    cancellation_token=ctx.cancellation_token,  
)
```



```
@event  
async def handle_request(self, message: GroupChatRequestPublish, ctx: MessageContext) -> None:  
    """Handle a content request event by passing the messages in the buffer  
    to the delegate agent and publish the response."""
```

Building Your own Multi-Agent System



- ◆ **Step2: Wrap the agent instance with DrSaiChatAgentContainer and assign the topic_ids to it.**

```
def _create_participant_factory(
    self,
    parent_topic_type: str,
    output_topic_type: str,
    agent: ChatAgent,
    message_factory: DrSaiMessageFactory,
    long_task_topic_type: str
) -> Callable[[], DrSaiChatAgentContainer]:
    def _factory() -> DrSaiChatAgentContainer:
        container = DrSaiChatAgentContainer(
            parent_topic_type,
            output_topic_type,
            agent,
            message_factory,
            long_task_topic_type,
        )
        return container
    return _factory
```

Pass in the topic_id of the DrSaiBaseGroupChatManager and other agents to enable sending GroupChat events later.

The agent instance

Send long-task information to objects subscribed to the corresponding long_task_topic_type.

Moreover, you can develop these functions about handling and sending/emitting various GroupChat events in the DrSaiChatAgentContainer.

DrSaiChatAgentContainer is located in:

python/packages/drsai/src/drsai/modules/groupchat/drsai_base_agent_container.py

Building Your own Multi-Agent System



- ◆ Step3: In DrSaiBaseGroupChat, organize the initialization information and assign the appropriate topic_id values that the GroupChatManager and each ChatAgentContainer need to receive or send.

```
class AGRoundRobinGroupChat(DrSaiBaseGroupChat):  
  
    def _create_group_chat_manager_factory(  
        self,  
        name: str,  
        group_topic_type: str,  
        output_topic_type: str,  
        participant_topic_types: List[str],  
        participant_names: List[str],  
        participant_descriptions: List[str],  
        output_message_queue: asyncio.Queue[BaseAgentEvent | BaseChatMessage | GroupChatTermination],  
        termination_condition: TerminationCondition | None,  
        max_turns: int | None,  
        message_factory: MessageFactory,  
    ) -> Callable[[], AGRoundRobinGroupChatManager]:  
  
        def _factory() -> AGRoundRobinGroupChatManager:  
            return AGRoundRobinGroupChatManager(  
                name,  
                group_topic_type,  
                output_topic_type,  
                participant_topic_types,  
                participant_names,  
                participant_descriptions,  
                output_message_queue,  
                termination_condition,  
                max_turns,  
                message_factory,  
                self._emit_team_events,  
                db_manager=self._db_manager,  
            )  
  
        return _factory  
  
    def _create_participant_factory(  
        self,  
        parent_topic_type: str,  
        output_topic_type: str,  
        agent: ChatAgent,  
        message_factory: DrSaiMessageFactory,  
        long_task_topic_type: str,  
    ) -> Callable[[], DrSaiChatAgentContainer]:  
  
        def _factory() -> DrSaiChatAgentContainer:  
            container = DrSaiChatAgentContainer(  
                parent_topic_type,  
                output_topic_type,  
                agent,  
                message_factory,  
                long_task_topic_type,  
            )  
  
            return container  
  
        return _factory
```

```
async def _init(self, runtime: AgentRuntime) -> None:  
    # Constants for the group chat manager.  
    group_chat_manager_agent_type = AgentType(self._group_chat_manager_topic_type)  
  
    # Register participants.  
    # Use the participant topic type as the agent type.  
    for participant, agent_type in zip(self._participants, self._participant_topic_types, strict=True):  
        # Register the participant factory.  
        await DrSaiChatAgentContainer.register(  
            runtime,  
            type=agent_type,  
            factory=self._create_participant_factory(  
                self._group_topic_type,  
                self._output_topic_type,  
                participant,  
                self._message_factory,  
                self._long_task_topic_type,  
            ),  
        )  
  
        # Add subscriptions for the participant.  
        # The participant should be able to receive messages from its own topic.  
        await runtime.add_subscription(TypeSubscription(topic_type=agent_type, agent_type=agent_type))  
        # The participant should be able to receive messages from the group topic.  
        await runtime.add_subscription(TypeSubscription(topic_type=self._group_topic_type, agent_type=agent_type))  
        # The participant should be able to receive messages from the long task topic.  
        await runtime.add_subscription(  
            TypeSubscription(topic_type=self._long_task_topic_type, agent_type=agent_type)  
        )  
  
    # Register the group chat manager.  
    await self._base_group_chat_manager_class.register(  
        runtime,  
        type=group_chat_manager_agent_type.type,  
        factory=self._create_group_chat_manager_factory(  
            name=self._group_chat_manager_name,  
            group_topic_type=self._group_topic_type,
```

Assign the appropriate topic_id values that the GroupChatManager and each ChatAgentContainer

Building Your own Multi-Agent System



- ◆ Step4: In DrSaiBaseGroupChat, send the "GroupChatStart" GroupChat event to start group chat.

```
class AGRoundRobinGroupChat(DrSaiBaseGroupChat):
```

"run_stream"

```
await self._runtime.send_message(  
    GroupChatStart(messages=messages),  
    recipient=AgentId(type=self._group_chat_manager_topic_type, key=self._team_id),  
    cancellation_token=cancellation_token,  
)
```

AgentRuntime

DrSaiBaseGroupChatManager



- ◆ **Round Robin GroupChat example is: `examples/agent_groupchat/groupchat_roundgroup.py`:**
 - **AGRoundRobinGroupChat:**
`python/packages/drsai/src/drsai/modules/groupchat/ag_round_robin_group_chat.py`
 - **AGRoundRobinGroupChatManager:**
`python/packages/drsai/src/drsai/modules/groupchat/ag_round_robin_group_chat.py`

- ◆ **GroupChat and Long Task example is in `examples/agent_groupchat/groupchat_task_by_AG`:**

05-How to Build Your Own Scientific AI Agent

When can we build AI Agent?

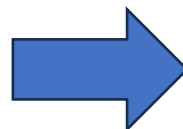


◆ TODO for Tools:

- ✓ Break the overall workflow down into atomic operations
- ✓ Determine whether tools in specialized execution environments can be exposed through MCP or OpenAPI interfaces
- ✓ ...

◆ Thinking about Your Work:

- ✓ Simple question–answering
- ✓ Only requires basic analysis via tool calls
- ✓ A relatively standard workflow
- ✓ Whether multi-part collaborative analysis is needed
- ✓ Whether human–computer interaction is required
- ✓ ...



◆ TODO for Specialized Knowledge :

- ✓ The domain knowledge that the large model needs to understand, such as specialized concepts and tool usage examples
- ✓ The necessary workflow details and procedural steps
- ✓ ...



Now, you can use OpenDrSai to develop scientific agents and applications.

When can we build AI Agent?

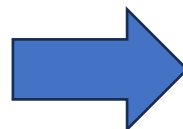


◆ Development for Operations:

- ✓ Convert each atomic operation into a **Python function, MCP, or OpenAPI** interface
- ✓ Connect tools in specialized execution environments through **MCP or OpenAPI**
- ✓ ...

◆ Work mode for Agent:

- ✓ Q&A and deep retrieval mode
- ✓ Tool-use and reflection mode
- ✓ Multi-agent collaboration mode
- ✓ Human-computer interaction mode
- ✓ ...



◆ Development for Specialized Knowledge :

- ✓ Use the **HEPAI knowledge platform and embedding models** for knowledge management
- ✓ ...

- OpenDrSai: <https://github.com/hepai-lab/drsai>
- HEPAI RAGFlow: <https://aiweb01.ihep.ac.cn:886/knowledge>
- HEPAI Models: <https://aiapi.ihep.ac.cn/>





Thank you for listening

Backup



◆ Access Mineru within HepAI: <https://ai.ihep.ac.cn/#/tools>

应用

探索我们的智能工具集合，为您的研究和工作提供强大的支持

HaiChat
智能对话助手，支持自然语言交互和问题解答
打开 →

Dr.Sai
物理分析智能体，专业的高能物理数据分析工具
打开 →

HaiAcademic
学术助手，帮助您进行文献查询和学术写作
打开 →

PDF解析器
智能PDF文档解析和提取工具
打开 →

主要功能

- ✓ 智能对话和问答
- ✓ 数据分析和可视化
- ✓ 文献查询和学术支持
- ✓ 文档处理和提取

使用提示

快速开始
点击任何工具卡片即可快速启动相应功能

最佳实践
根据您的具体需求选择合适的工具，获得最佳体验

PDF 文件上传与解析
支持批量上传、智能解析、实时预览

解析模式
A PDF parser for MinerU v1.0

拖拽 PDF 到此，或 [点击选择文件](#)
支持单个 ≤ 20MB，PDF 格式

文件列表 (0) [开始解析](#)

实时解析结果
[全部结果](#) [解析中](#) [已完成](#)

暂无实时解析结果
上传 PDF 开始解析，或前往文件管理查看历史结果
[文件管理](#)

开发者指南
MinerU API 使用示例与代码参考

[快速开始](#)
[安装依赖](#)
[环境配置](#)
> 代码示例
[PDF 文件解析](#)
[PPTX 文件解析](#)
[DOCX 文件解析](#)
[图片文件解析](#)
[Base64 编码文件](#)

PDF 文件解析
解析 PDF 文件并转换为 Markdown 格式

```
from hepai import HRModel, HepAI
import base64, os
from pathlib import Path

# Step 1: 上传文件到服务器
client = HepAI(
    api_key=os.environ["HEPAI_API_KEY"],
    base_url="https://aiapi.ihep.ac.cn/apiv2"
)

# PDF 文件路径
pdf_file_path = '/path/to/your/file.pdf'
file_path = pdf_file_path

file_obj = client.files.create(
    file=open(pdf_file_path, "rb"),
    purpose="user_data"
)

# Step 2: 调用 MinerU 模型进行解析
model = HRModel.connect(
    api_key=os.environ["HEPAI_API_KEY"],
    name="hepai/mineru2_0.98",
    base_url="https://aiapi.ihep.ac.cn/apiv2"
```

使用步骤:

1. 确保已安装 HepAI Python SDK
2. 设置 HEPAI_API_KEY 环境变量
3. 修改文件路径为您的实际文件路径
4. 运行脚本，等待解析完成

支持的文件格式:
PDF、PPTX、DOCX、PNG、JPG、JPEG 等多种格式